



Московский государственный университет имени М.В.Ломоносова
Факультет вычислительной математики и кибернетики
Кафедра алгоритмических языков

Моисеев Дмитрий Александрович

Разработка и адаптация методов оценки
RAG-систем (систем генерации, дополненной поиском)
для русского языка

Development and adaptation of evaluation
methods for retrieval augmented generation systems
in Russian

МАГИСТЕРСКАЯ ДИССЕРТАЦИЯ

Научный руководитель:
Канд. физ.-мат. наук
М.М. Тихомиров

Москва, 2026

Аннотация

Разработка и адаптация методов оценки
RAG-систем (систем генерации, дополненной поиском)
для русского языка

Моисеев Дмитрий Александрович

В работе предложен и реализован комплексный подход к оценке качества русскоязычных Retrieval-Augmented Generation систем. На базе англоязычного бенчмарка RAGBench собран и опубликован его русскоязычный аналог RAGBench-RU: текстовые поля 7 поддатасетов переведены моделью Qwen2.5-72B-Instruct, а эталонная разметка перенесена из исходного RAGBench по сохранённым ключам предложений без повторной автоматической аннотации. Разработан конвейер обучения токено-уровневых моделей-оценщиков TRACe с тремя классификационными головами (relevance, utilization, adherence) и автоматическим подбором порогов. Проведена основная сетка экспериментов на трёх базовых моделях (DeBERTa-v3-large, BGE-M3, RuModernBERT); ruRoPEBert-e5-base-2k проверялась только на предварительном этапе и была исключена из основной сетки. Варьировались семь длин контекста (128–8192 токенов), две схемы весов функции потерь (1:1:1 и 3:3:1), два языка (RU/EN) и две архитектуры головы (Simple, Complex) - всего около восьмидесяти обученных конфигураций. Лучшая конфигурация на русском (ModernBERT-base, длина 2048, Simple 3:3:1) достигает среднего Test F1 = 0.858 относительно эталонной разметки RAGBench-RU. Дополнительно Qwen2.5-72B использовалась как LLM-оценщик без дообучения на исходном англоязычном наборе данных RAGBench. Это сопоставление рассматривается как опорный ориентир, а не как прямое сравнение: русскоязычный RAGBench-RU на вход модели не подавался, и единая токен-уровневая метрика на одном корпусе не вычислялась. Результаты согласуются с идеей RAGBench о том, что компактный дообученный классификатор может быть практичной и более дешёвой альтернативой LLM-оценщику. Для демонстрации практического применения разработан Streamlit-сервис интерактивной оценки, а обученная мо-

дель дополнительно проверена на внешнем русскоязычном датасете SberQuAD-QA.

Abstract

Development and adaptation of evaluation
methods for retrieval augmented generation systems
in Russian

Moiseev Dmitry Alexandrovich

This thesis presents a comprehensive framework for evaluating Russian-language Retrieval-Augmented Generation systems. Building on the English RAGBench benchmark, we translate and release RAGBench-RU: textual fields of 7 sub-datasets are machine-translated with Qwen2.5-72B-Instruct, while the reference annotation is transferred from the original RAGBench via preserved sentence keys, without re-running automatic annotation. We design a training pipeline for token-level TRACe evaluators with three classification heads (relevance, utilization, adherence) and automatic threshold tuning. The main grid of experiments spans three base models (DeBERTa-v3-large, BGE-M3, RuModernBERT); ruRoPEBert-e5-base-2k was used only in a preliminary experiment and excluded from the main grid. The grid covers seven context lengths (128–8192 tokens), two loss-weight schemes (1:1:1 and 3:3:1), two languages (RU/EN) and two head architectures (Simple, Complex) - around eighty trained configurations in total. The best Russian configuration (ModernBERT-base, length 2048, Simple 3:3:1) reaches mean Test F1 = 0.858 against the RAGBench-RU reference annotation. In addition, Qwen2.5-72B was used as a zero-shot LLM evaluator on the original English RAGBench. This experiment is treated as a reference point rather than a direct comparison: RAGBench-RU examples were not passed to the model, and no shared token-level metric was computed on a single corpus. The results are nevertheless consistent with the central idea of RAGBench: a compact fine-tuned classifier can be a practical and cheaper alternative to an LLM evaluator. To demonstrate practical use, we also developed an interactive Streamlit evaluation service and tested the trained model on the external

Russian SberQuAD-QA dataset.

Содержание

1	Введение	8
2	Обзор методов оценки RAG-систем	13
2.1	RAGAS	13
2.2	RAGBench и фреймворк TRACe	14
2.3	Luna	16
2.4	ARES	17
2.5	RAGTruth	18
2.6	AttributionBench	18
2.7	Сравнение фреймворков	19
3	Постановка задачи	22
3.1	Формальная постановка задачи классификации	22
3.2	Переход от токенных предсказаний к агрегированным метрикам	23
3.3	Метрики качества модели-оценщика	23
3.4	Задачи, решаемые в диссертации	24
4	Русскоязычный набор данных RAGBench-RU	26
4.1	Исходный RAGBench и отбор поднаборов	26
4.2	Схема одного примера	26
4.3	Схема перевода корпуса	27
4.4	Контроль качества перевода и перенос разметки	28
4.5	Статистика корпуса	30
4.6	Публикация	31
5	Архитектура модели-оценщика	32
5.1	Общая схема	32
5.2	Архитектуры выходных классификационных блоков	32
5.3	Функция потерь	33
5.4	Подбор порогов	34
5.5	Выбор базовой модели	34

5.6	Предварительный эксперимент с ruRoPEBert	35
5.7	Детали обучения	37
6	Экспериментальная часть	39
6.1	Краткая сводка по экспериментам	39
6.2	Сетка моделей и длин контекста	40
6.3	Анализ влияния весов функции потерь	42
6.4	Сравнение Simple и Complex	43
6.5	Сравнение RU и EN	46
6.6	Подбор порогов	47
6.7	Классификатор vs LLM-оценщик	48
6.8	Корреляция со значениями TRACe-метрик	51
6.9	Применение на внешнем наборе данных SberQuAD-QA	51
6.10	Выводы по экспериментальной главе	54
7	Интерактивный сервис оценки	55
7.1	Назначение	55
7.2	Архитектура	55
7.3	Основные возможности	56
7.4	Ограничения	57
8	Заключение	59
8.1	Полученные результаты	59
8.2	Границы применимости и дальнейшее развитие	61
8.3	Артефакты работы	61
	Список литературы	62
	Приложение	65

1 Введение

Генерация, дополненная поиском, (Retrieval-Augmented Generation, RAG) [1] представляет собой подход в задачах обработки естественного языка, сочетающий информационный поиск и генерацию текста. Вместо того чтобы опираться только на знания, заложенные в параметрах языковой модели, RAG-система сначала извлекает внешние документы, а затем использует их при формировании ответа. Такой подход особенно востребован в прикладных задачах, где важны актуальность ответа, опора на источники и возможность работать с большими корпусами документов.

Однако повышение качества отдельных компонентов RAG-системы не всегда приводит к улучшению ее работы в целом. Более релевантное извлечение документов само по себе не гарантирует формирования корректного ответа, а увеличение полноты генерации может сопровождаться снижением фактической точности, некорректной атрибуцией источников или появлением галлюцинаций. В связи с этим оценка RAG-систем требует не только раздельного анализа компонентов поиска и генерации, но и подхода, позволяющего учитывать их взаимодействие и влияние на качество итогового ответа.

В последние годы для этой задачи были предложены как LLM-ориентированные фреймворки оценки, так и обучаемые модели-оценщики. Наиболее известными примерами являются RAGAS [2], RAGBench [3], Luna [4], ARES [5], RAGTruth [6] и AttributionBench [7]. Эти работы предлагают разные способы измерять качество ответа, полноту использования контекста и степень его согласованности с источниками. Среди них особое место занимает RAGBench, поскольку он объединяет крупный открытый корпус и интерпретируемую схему разметки TRACe. По результатам оригинальной статьи [3], компактный дообученный классификатор может превосходить большую языковую модель без дообучения [8] по стабильности и точности на англоязычных поднаборах. Этот результат относится к оригинальной работе и в настоящей диссертации не воспроизводится в виде прямого сравнения по единой метрике на одном корпусе, а используется как методологический ориентир.

Для русского языка ситуация остается существенно менее изученной. Несмотря на рост числа русскоязычных RAG-приложений, отсутствует общепринятый открытый бенчмарк, совместимый с TRACe-подходом, а также систематический анализ того, как русскоязычные и мультиязычные базовые модели ведут себя в задаче токен-уровневой

оценки RAG. Это затрудняет как сравнение систем между собой, так и перенос результатов англоязычных работ на русский язык.

Настоящая работа посвящена воспроизводимому переносу подхода RAGBench/TRACe на русский язык. В рамках работы создан русскоязычный бенчмарк, обучено семейство моделей-оценщиков и проведена серия экспериментов для сравнения архитектур и анализа зависимости метрик от длины контекста, схемы весов функции потерь и языка.

Цель и задачи работы

Цель данной диссертации состоит в разработке и экспериментальной проверке воспроизводимого подхода к оценке русскоязычных RAG-систем, включающего русскоязычный бенчмарк, семейство моделей-оценщиков и средства их практического применения.

Для достижения этой цели решаются следующие задачи:

1. построить и опубликовать русскоязычный бенчмарк RAGBench-RU на основе 7 поднаборов исходного RAGBench, переведенных с помощью LLM;
2. воспроизвести схему оценки без дообучения на Qwen2.5-72B по аналогии с протоколом исходного RAGBench на тестовых поднаборах англоязычного корпуса (см. приложение А.3) и использовать полученные результаты как опорный ориентир;
3. обучить семейство моделей-оценщиков TRACe, работающих на уровне токенов, варьируя базовую модель, длину контекста, веса функции потерь и архитектуру классификационного слоя;
4. провести сравнительный анализ полученных конфигураций: исследовать зависимость метрик от длины контекста, влияние схемы весов функции потерь и архитектуры выходного классификационного блока, сравнить модели, обученные на русскоязычной и англоязычной версиях корпуса, а также сопоставить результаты обученного классификатора с опорной оценкой большой языковой модели без дообучения;

5. реализовать интерактивный сервис, позволяющий применять обученные модели к произвольным тройкам «вопрос – контекст – ответ».

При этом сопоставление с оценщиком на базе большой языковой модели без дообучения не является прямым сравнением по единой метрике на одном корпусе: Qwen2.5-72B применялась к исходному англоязычному набору RAGBench, тогда как обученные классификаторы оценивались на RAGBench-RU.

Положения, выносимые на защиту

1. Построен и опубликован русскоязычный бенчмарк RAGBench-RU (HuggingFace <https://huggingface.co/datasets/CMCenjoyer/ragbench-ru>), полученный систематическим переводом 7 поднаборов исходного RAGBench с сохранением структуры разметки TRACe.
2. Семейство русскоязычных токен-уровневых моделей-оценщиков позволяет с высокой точностью предсказывать метки relevance, utilization и adherence; лучшая конфигурация достигает среднего Test F1 = 0.858 на объединенном тестовом корпусе.
3. Воспроизведена и количественно охарактеризована схема оценки без дообучения на базе большой языковой модели Qwen2.5-72B на исходном англоязычном RAGBench: средняя доля совпадений по предложениям составляет 0.70 для релевантности и 0.83 для использования контекста, а также выявлена устойчивая склонность к избыточной разметке (в среднем около 1.28 лишних релевантных ключей и 0.73 лишних использованных ключей на пример), наиболее выраженная в юридическом и научном поднаборах. Эти значения используются в работе как опорный ориентир и согласуются с ключевой идеей оригинальной статьи RAGBench о практической применимости локального классификатора как более дешёвой замены большой языковой модели; при этом прямое сопоставление двух методов по единой метрике на уровне токенов и на одном корпусе в настоящей работе не проводится и относится к направлениям дальнейшего исследования.
4. На исследованной выборке конфигураций простая классификационная архитектура систематически оказывается устойчивее сложной, а оптимальная длина кон-

текста для лучших моделей лежит в диапазоне порядка 1024–4096 токенов с максимумом при 2048.

5. Разработан прикладной Streamlit-сервис, позволяющий использовать обученные оценщики для одиночной и пакетной оценки новых примеров без повторного обучения.

Научная новизна и практическая значимость

Научная новизна. Данная работа представляет одну из первых систематических попыток подхода RAGBench/TRACe на русский язык, объединяющих публичный бенчмарк, обученные модели-оценщики и анализ факторов, влияющих на качество токенов-уровневой оценки. В отличие от подходов, основанных преимущественно на вызовах LLM в стиле RAGAS или на закрытых внутренних метриках отдельных команд, в работе предлагается воспроизводимая исследовательская схема, доступная для дальнейшего сравнения и развития.

Практическая значимость. Полученные результаты позволяют: (а) сравнивать русскоязычные RAG-системы в единой методологии; (б) использовать обученные оценщики как локальную и существенно более дешевую альтернативу LLM-оценке в автономных сценариях; (в) применять разработанные модели к новым данным через открытый обучающий конвейер и интерактивный сервис.

Апробация результатов

Основные результаты настоящей работы были представлены и обсуждены на научной конференции «Ломоносовские чтения - 2026» (секция «Вычислительная математика и кибернетика»), МГУ имени М. В. Ломоносова, 23 марта - 5 апреля 2026 г. По теме настоящей работы опубликованы тезисы доклада [9].

Структура работы

Раздел 2 содержит обзор существующих подходов к оценке RAG-систем и обосновывает выбор RAGBench/TRACe в качестве методологической основы работы. В разделе 3 формализуется постановка задачи. Раздел 4 описывает построение корпуса

RAGBench-RU, конвейер перевода и контроль качества. В разделе 5 излагается архитектура модели-оценщика, варианты выходного классификационного блока и процедура подбора порогов. Раздел 6 посвящен экспериментальному анализу: сравнению моделей, длин контекста, весов функции потерь, языков разметки и LLM-оценки. В разделе 7 описан интерактивный сервис применения обученных моделей. В разделе 8 подводятся итоги, обсуждаются ограничения и направления дальнейшей работы.

2 Обзор методов оценки RAG-систем

В данном разделе рассматриваются современные подходы, фреймворки и модели, применяемые для автоматической оценки RAG-систем. Цель раздела — показать, на каких методологических принципах основан предлагаемый в настоящей работе подход, а также обосновать выбор RAGBench/TRACe в качестве основы для русскоязычной адаптации.

2.1 RAGAS

RAGAS (Retrieval Augmented Generation Assessment) [2] - один из первых автоматических фреймворков оценки RAG. Он использует LLM (в оригинале - GPT-3.5/4) для вычисления трёх метрик: *Faithfulness* (достоверность ответа относительно контекста), *Answer Relevance* (релевантность ответа вопросу) и *Context Relevance* (релевантность извлечённого контекста вопросу).

Faithfulness. Ответ A разбивается LLM на атомарные утверждения $S = \{s_1, \dots, s_n\}$; для каждого s_i новым запросом проверяется, поддерживается ли оно текстом контекста $c(q)$. Метрика:

$$F = \frac{|V|}{|S|}, \quad |V| = \sum_{i=1}^{|S|} v(s_i, c(q)).$$

Answer Relevance. Из ответа A генерируется набор вопросов $\{q_i\}_{i=1}^n$, и считается средняя косинусная близость между исходным вопросом q и каждым q_i в пространстве эмбедингов:

$$AR = \frac{1}{n} \sum_{i=1}^n \cos(\phi(q), \phi(q_i)).$$

Context Relevance. LLM извлекает из контекста подмножество релевантных предложений S_{ext} ; метрика - отношение $|S_{\text{ext}}|/|\text{sentences}(c(q))|$.

Преимущества. RAGAS не требует ручной разметки эталонных оценок, имеет модульную структуру и поддерживает интеграцию с различными LLM, эмбединговыми моделями и ретриверами. Благодаря этому фреймворк может быть адаптирован к разным доменам и языкам при наличии подходящих моделей для генерации, оценки и построения эмбедингов.

Ограничения. Основные ограничения RAGAS связаны с зависимостью значительной части метрик от внешнего LLM-оценщика, что повышает стоимость вычислений и может снижать воспроизводимость результатов при изменении модели, промпта или параметров генерации. Для русского языка применение RAGAS требует использования мультязычных или русскоязычных LLM и эмбединговых моделей; при этом отсутствует общепринятая русскоязычная валидационная разметка, позволяющая независимо оценить надёжность получаемых метрик.

2.2 RAGBench и фреймворк TRACe

RAGBench [3] - масштабный бенчмарк из 100 тыс. примеров в 12 поднаборах, покрывающих пять доменов: биомедицина (PubMedQA, CovidQA), общие знания (HotpotQA, MS Marco, HAGRID, ExpertQA), юридические контракты (CuAD), клиентская поддержка (DelucionQA, EManual, TechQA) и финансы (FinQA, TAT-QA). Каждый пример размечен четырьмя метриками фреймворка TRACe:

1. Relevance. Доля релевантных токенов в документах контекста относительно общего числа:

$$\text{relevance} = \frac{\sum_i |R_i|}{\sum_i |d_i|},$$

где R_i - множество релевантных токенов документа d_i .

2. Utilization. Доля использованных в ответе токенов контекста:

$$\text{utilization} = \frac{\sum_i |U_i|}{\sum_i |d_i|}.$$

3. Completeness. Доля релевантных токенов, которые реально были использованы:

$$\text{completeness} = \frac{\sum_i |R_i \cap U_i|}{\sum_i |R_i|}.$$

4. Adherence. Булева метрика: все ли утверждения в ответе подтверждаются контекстом. Позволяет обнаруживать галлюцинации.

Разметка формируется с использованием LLM-аннотатора на основе GPT-4: для каждого примера модель выделяет релевантные предложения в документах, определяет фрагменты, использованные при формировании ответа, а также проверяет, подтверждается ли каждое предложение ответа предоставленным контекстом. Валидация на синтетических данных KILT с известными эталонными оценками показала высокую

ранговую корреляцию по коэффициенту Кендалла [10], что авторы рассматривают как подтверждение надёжности автоматической разметки.

Важной методологической особенностью RAGBench является сравнение различных оценщиков на одном и том же наборе данных (рис. 1). Согласно результатам авторов оригинальной статьи [3], дообученная DeBERTa-v3-large [11] предсказывает TRACe-метрики на англоязычных поднаборах точнее, чем большая языковая модель без дообучения [8] в схожем формате запроса, а также фреймворки RAGAS и TruLens. Это наблюдение относится к работе авторов RAGBench и в настоящей диссертации напрямую не воспроизводится; оно рассматривается как методологический ориентир, обосновывающий направление, при котором локальная специализированная модель может выступать более дешёвой заменой вызовам большой языковой модели.

Dataset	GPT-3.5			RAGAS			TruLens			DeBERTA		
	Hal↑	Rel↓	Util↓	Hal↑	Rel↓	Util↓	Hal↑	Rel↓	Util↓	Hal↑	Rel↓	Util↓
PubMedQA	0.51	0.21*	0.16	0.54	0.37	-	0.62	0.45	-	0.80*	0.26	0.17
CovidQA-RAG	0.57	0.18	0.11	0.58	0.17	-	0.62	0.58	-	0.77*	0.19	0.11
HotpotQA	0.59	0.11	0.08	0.62	0.14	-	0.64	0.73	-	0.85*	0.11	0.08
MS Marco	0.65	0.23	0.11	0.63	0.25	-	0.62	0.61	-	0.70	0.22	0.10
HAGRID	0.58	0.22	0.15	0.62	0.22	-	0.67	0.69	-	0.81*	0.20*	0.13
ExpertQA	0.55	0.31	0.23	0.57	0.28	-	0.70	0.60	-	0.87*	0.18*	0.11*
DelucionQA	0.57	0.18	0.10	0.70*	0.22	-	0.55	0.64	-	0.64	0.15*	0.10
EManual	0.54	0.17	0.11*	0.57	0.27	-	0.61	0.64	-	0.76*	0.13*	0.13
TechQA	0.51	0.10	0.05	0.52	0.12	-	0.57	0.70	-	0.86*	0.08*	0.04*
FinQA	0.57	0.10	0.13	0.57	0.06*	-	0.53	0.79	-	0.81*	0.10	0.10
TAT-QA	0.52	0.20	0.17*	0.63	0.18*	-	0.59	0.72	-	0.83*	0.27	0.23
CUAD	0.51	0.27	0.11	0.66	0.19*	-	0.40	0.66	-	0.80*	0.24	0.10

Рис. 1: Сравнение методов оценки на тестовых выборках RAGBench [3]

Преимущества. RAGBench представляет собой масштабный и доменно - диверсифицированный корпус, содержащий интерпретируемый набор токен-уровневых метрик. Дополнительным преимуществом является демонстрация того, что дообучение специализированного оценщика позволяет существенно повысить качество автоматической оценки по сравнению с подходами без дообучения.

Ограничения. Основными ограничениями RAGBench являются ориентация на английский язык, отсутствие отдельных метрик отказа от ответа и безопасности, а также зависимость качества разметки от LLM-аннотатора на основе GPT-4.

2.3 Luna

Luna [4] — специализированный оценщик галлюцинаций в RAG-системах, предназначенный для промышленного применения. Модель основана на энкодере DeBERTa-v3-Large с 440 млн параметров, дообученном определять, какие токены ответа подтверждаются переданным контекстом. В качестве начальной точки используется версия модели, предварительно обученная на задаче вывода на естественном языке (Natural Language Inference, NLI), где требуется определять отношение между двумя текстами: логическое следование, противоречие или нейтральность. Голова предсказания инициализируется весами NLI-классификатора, что позволяет перенести эту постановку на задачу токенов-уровневой проверки ответа по контексту.

Для обработки длинного контекста вход разбивается на окна длиной L , каждое из которых содержит часть контекста, вопрос и полный ответ:

$$w_i = [c_{i_1} \dots c_{i_l}] \oplus [q_1 \dots q_Q] \oplus [r_1 \dots r_R],$$

где $l = L - Q - R$. Для каждого окна модель предсказывает вероятности поддержки токенов ответа:

$$P_S(w_i) = [p_1^i, \dots, p_R^i].$$

На этапе применения модели для каждого токена ответа выбирается максимальная вероятность поддержки среди всех окон:

$$p_j = \max_i p_j^i,$$

после чего вероятность поддержки всего ответа определяется как минимальная вероятность среди токенов:

$$P_S = \min_j p_j, \quad P_H = 1 - P_S,$$

где P_H трактуется как вероятность галлюцинации ответа.

Экспериментальные результаты показывают, что Luna превосходит оценщики на базе GPT-3.5, а также фреймворки RAGAS и TruLens, в задаче обнаружения галлюцинаций при существенно меньших вычислительных затратах и меньшем времени получения оценки. На наборе RAGTruth модель показывает более высокое качество по сравнению с подходами, основанными на прямых запросах к большим языковым моделям, в задачах вопросно-ответной генерации и автоматического резюмирования. На внутреннем

тестовом наборе RAG QA Luna также превосходит рассмотренные базовые оценщики во всех предметных направлениях. Таким образом, Luna демонстрирует практическую применимость подхода, при котором специализированная локальная модель может выступать менее затратной и более быстрой альтернативой оценщику на основе большой языковой модели.

Отдельно следует отметить связь Luna с RAGBench/TRACe. Хотя в статье, посвящённой RAGBench, явная ссылка на Luna отсутствует, в личной переписке с авторами RAGBench было уточнено, что при построении RAGBench использовались отдельные инженерные решения, близкие к Luna, включая оконное разбиение длинного контекста и аугментации документов. В связи с этим Luna представляет интерес не только как самостоятельный оценщик галлюцинаций, но и как пример инженерной реализации подходов, применимых при построении специализированного локального оценщика RAG-систем.

2.4 ARES

ARES [5] предлагает автоматизированный подход к оценке RAG-систем, основанный на обучении специализированных моделей-оценщиков на синтетических данных. В качестве основных критериев рассматриваются релевантность найденного контекста запросу, обоснованность ответа предоставленным контекстом и релевантность ответа исходному вопросу. На первом этапе большая языковая модель генерирует синтетические тройки «запрос – фрагмент контекста – ответ», включающие как положительные, так и отрицательные примеры. Отрицательные примеры формируются не только за счёт некорректных или противоречивых ответов, но и за счёт нерелевантных фрагментов контекста и ответов, не соответствующих исходному запросу. Затем на этих данных дообучаются отдельные модели-оценщики для каждого из рассматриваемых критериев, например на основе DeBERTa-v3-Large.

Важной особенностью ARES является сочетание синтетического обучения с небольшим объёмом ручной разметки: для повышения надёжности итоговой оценки авторы используют набор человеческих аннотаций и метод prediction-powered inference, позволяющий корректировать ошибки модели-оценщика и строить доверительные интервалы для сравнения RAG-систем. Тем самым ARES не полностью исключает ручную разметку, но существенно снижает её необходимый объём.

Преимуществом ARES является возможность получать специализированные оценщики для конкретного домена без формирования крупного вручную размеченного корпуса. Ограничение подхода связано с тем, что качество обучаемого оценщика зависит от распределения и качества синтетически сгенерированных примеров, а также от репрезентативности небольшого валидационного набора с ручной разметкой.

В контексте настоящей работы идея ARES используется частично: большая языковая модель Qwen2.5-72B [12] применяется для перевода исходного корпуса и в качестве LLM-оценщика без дообучения для сравнения, однако не выступает в качестве финального оценщика. Финальная оценка, как и в направлении ARES/RAGBench, переносится на специализированную локальную модель, что позволяет снизить стоимость и повысить воспроизводимость автоматической оценки.

2.5 RAGTruth

RAGTruth [6] - корпус почти из 18 тыс. ответов, сгенерированных RAG-системами и вручную размеченных для задачи обнаружения галлюцинаций. Разметка выполнена как на уровне всего ответа, так и на уровне отдельных слов, что позволяет выделять неподтверждённые или противоречащие контексту фрагменты.

Сильной стороной RAGTruth является высокое качество ручной разметки и более детальный анализ галлюцинаций по сравнению с оценкой только на уровне всего ответа. Вместе с тем корпус не содержит разметки в схеме TRACe: в нём отсутствуют отдельные токен-уровневые метки релевантности контекста и использования контекста в ответе. Поэтому в настоящей работе RAGTruth не используется как обучающий корпус, однако служит полезным ориентиром при анализе ошибок LLM-оценщика.

2.6 AttributionBench

AttributionBench [7] фокусируется на задаче автоматической оценки атрибуции ответа источнику, то есть проверки того, подтверждается ли каждое утверждение в сгенерированном ответе приведённым доказательством. Авторы показывают, что эта задача остаётся сложной даже для современных больших языковых моделей: качество GPT-4 и GPT-3.5 заметно зависит от постановки запроса, а простое изменение промпта не решает проблему надёжной оценки.

Основной вывод работы состоит в том, что атрибуцию ответа к источнику трудно надёжно оценивать одним запросом к LLM без дополнительной настройки. Это поддерживает общий вывод настоящей работы о необходимости специализированных оценщиков, в том числе моделей, обучаемых для проверки связи между контекстом и отдельными фрагментами ответа.

2.7 Сравнение фреймворков

Сводная характеристика рассмотренных фреймворков приведена в таблице 1. Для настоящей работы RAGBench/TRACe выбран в качестве базы по совокупности признаков: наличие публично доступного корпуса достаточного объёма, интерпретируемые метрики на уровне токенов, эмпирически обоснованная стратегия обучения компактного оценщика, и возможность использовать LLM-аннотатор только на этапе разметки (а не во время применения).

Таблица 1: Сравнение рассмотренных подходов к оценке RAG-систем

Подход / ресурс	Тип оценки	Разметка	Уровень	Корпус	Язык
RAGAS [2]	LLM-оценщик	не требуется	ответ / контекст	нет	EN / мульти.
ARES [5]	обучаемый оценщик	синтетическая и малая ручная	пример	нет	EN
RAGTruth [6]	корпус для анализа галлюцинаций	ручная	ответ и слова	да	EN
Attribution- Bench [7]	корпус для оценки атрибуции	ручная	утверждения	да	EN
Luna [4]	обучаемый оценщик	NLI + RAG-данные	токены ответа	частично	EN
RAGBench [3]	обучаемый оценщик + LLM-аннотатор	GPT-4	токены	да	EN
RAGBench- RU (настоящая работа)	обучаемый оценщик + LLM-ориентир	перенос разметки после перевода	токены	да	RU

Вывод. Проведённое сравнение показывает, что существующие подходы решают разные аспекты задачи оценки RAG-систем. RAGAS удобен как готовый инструмент без необходимости предварительной разметки, ARES показывает возможность обучения оценщиков на синтетических данных, RAGTruth и AttributionBench дают качественные ручные корпуса для анализа галлюцинаций и атрибуции, а Luna демонстрирует практическую эффективность локального обучаемого оценщика.

При этом наиболее подходящей отправной точкой для настоящей работы является RAGBench/TRACe, поскольку он сочетает открытый корпус, интерпретируемую токеновую схему разметки и возможность перенести финальную оценку с большой языковой модели на специализированный локальный классификатор. Именно это делает RAGBench/TRACe удобной основой для русскоязычной адаптации. В следующем раз-

деле формализуется задача, решаемая в настоящей работе.

3 Постановка задачи

В настоящей работе разрабатывается комплекс средств для автоматической оценки русскоязычных RAG-систем. Формально эта задача включает три подзадачи: (а) сбор русскоязычного размеченного корпуса, (б) обучение модели-оценщика, (в) анализ поведения обученной модели при варьировании базовой архитектуры, длины контекста, весов функции потерь и выходного классификационного блока, а также сопоставление полученных результатов с конфигурациями, обученными на англоязычной разметке, и с LLM-оценщиком без дообучения как опорным ориентиром.

3.1 Формальная постановка задачи классификации

Пусть q - вопрос пользователя, $D = (d_1, \dots, d_{|D|})$ - упорядоченный набор извлечённых ретривером документов, a - ответ генератора. Модель-оценщик M_θ получает на вход объединённую последовательность

$$x = [q] [\text{SEP}] [d_1, \dots, d_{|D|}] [\text{SEP}] [a],$$

разбитую на токены $x = (x_1, \dots, x_T)$, и для каждого токена возвращает три вещественные оценки:

$$M_\theta(x) = (\ell_t^{(R)}, \ell_t^{(U)}, \ell_t^{(A)})_{t=1}^T.$$

Здесь $\ell_t^{(R)}$, $\ell_t^{(U)}$ и $\ell_t^{(A)}$ обозначают значения до сигмоидного преобразования для трёх целевых признаков TRACe.

Три выходных классификационных слоя соответствуют следующим целевым меткам TRACe:

- **Relevance:** бинарная метка на каждом токене документов $y_t^{(R)} \in \{0, 1\}$. Равна 1, если токен входит в предложение, помеченное LLM-аннотатором как релевантное запросу.
- **Utilization:** бинарная метка на каждом токене документов $y_t^{(U)} \in \{0, 1\}$. Равна 1, если токен входит в предложение, использованное в ответе.
- **Adherence:** булева метка поддержки всего ответа контекстом. В реализации она представляется как одинаковая бинарная метка $y_t^{(A)} \in \{0, 1\}$ на всех токенах ответа: 1, если ответ в целом поддержан контекстом, и 0 в противном случае.

Введём две бинарные маски: $m_t^{(C)}$ - маска токенов документов (для $y^{(R)}$ и $y^{(U)}$) и $m_t^{(A)}$ - маска токенов ответа (для $y^{(A)}$). Тогда функция потерь задаётся как нормированная взвешенная сумма бинарных кросс-энтропий по валидным токенам:

$$\mathcal{L}(\theta) = \frac{\alpha_R \mathcal{L}_R + \alpha_U \mathcal{L}_U + \alpha_A \mathcal{L}_A}{\alpha_R + \alpha_U + \alpha_A},$$

$$\mathcal{L}_R = \frac{\sum_t m_t^{(C)} \cdot \text{BCE}(\sigma(\ell_t^{(R)}), y_t^{(R)})}{\sum_t m_t^{(C)}},$$

и аналогично для $\mathcal{L}_U$, $\mathcal{L}_A$ (последний - по маске $m_t^{(A)}$). Нормализация на сумму весов сохраняет сопоставимый масштаб функции потерь при переходе между разными схемами $(\alpha_R, \alpha_U, \alpha_A)$.

В работе рассматриваются две схемы весов: $(\alpha_R, \alpha_U, \alpha_A) = (1, 1, 1)$ (базовая) и $(3, 3, 1)$ (гипотеза о том, что adherence - более лёгкая задача и не требует увеличенного веса).

3.2 Переход от токенных предсказаний к агрегированным метрикам

При применении модели бинаризация производится по порогам (τ_R, τ_U, τ_A) :

$$\hat{y}_t^{(R)} = \mathbb{I}[\sigma(\ell_t^{(R)}) \geq \tau_R].$$

Для каждой конфигурации пороги подбираются независимо на валидационной выборке путём сеточного поиска по $\{0.1, 0.2, \dots, 0.7\}$, максимизируя F1 на уровне токенов (token-level F1) на соответствующей маске.

Итоговые TRACe-величины восстанавливаются агрегированием токенных предсказаний в рамках документа и примера (формулы из раздела 2.2); в работе, однако, основной метрикой качества модели-оценщика выступает именно F1 на уровне токенов по каждой целевой метке, так как он не зависит от ошибок, накапливающихся при агрегации.

3.3 Метрики качества модели-оценщика

Для сравнения конфигураций используются:

- Precision / Recall / F1 на уровне токенов по каждой из трёх целевых меток;

- средний F1 по трём целевым меткам: $\bar{F}_1 = (F_1^{(R)} + F_1^{(U)} + F_1^{(A)})/3$ - основной агрегат для ранжирования;
- корреляция Пирсона между предсказанными значениями TRACe-метрик на уровне примера и разметкой RAGBench - дополнительная сверка с оригинальной схемой.

3.4 Задачи, решаемые в диссертации

Исходя из постановки, работа состоит из следующих задач:

1. **LLM-оценщик без дообучения (раздел 6.7).** Воспроизвести схему оценки без дообучения на модели Qwen2.5-72B по аналогии с оригинальной статьёй RAGBench и использовать полученные результаты в качестве опорного варианта сравнения с обученным классификатором; зафиксировать долю формальных ошибок (приложение А.3). При этом LLM-оценщик запускался на тестовых поднаборах исходного англоязычного RAGBench (см. приложение А.3) с оригинальным англоязычным промптом из статьи; русскоязычные данные RAGBench-RU на вход большой языковой модели не подавались. Поэтому сопоставление с классификатором, обученным на RAGBench-RU, носит характер сравнения с опорным ориентиром, а не контролируемого эксперимента на одном корпусе.
2. **Русскоязычный корпус (раздел 4).** Построить RAGBench-RU, переведя 7 поднаборов исходного корпуса, провести контроль качества и опубликовать на HuggingFace.
3. **Обучение моделей-оценщиков (разделы 5–6.2).** Обучить модели на основе трёх базовых архитектур при нескольких длинах контекста, двух наборах весов функции потерь и двух типах выходного классификационного блока. Подбор гиперпараметров (скорость обучения для базовой части модели и выходного блока, разогрев скорости обучения, Dropout) фиксирован для всех запусков.
4. **Сравнительный анализ (подразделы 6.3–6.7).** Оценить влияние длины контекста, весов функции потерь, архитектуры выходного классификационного блока и языка; сопоставить обученный классификатор и LLM-оценщик без дообучения,

явно разделяя метрики уровня предложений для LLM-оценщика (совпадение по предложениям, среднее число пропусков) и F1 на уровне токенов для классификатора относительно эталонной разметки RAGBench-RU. При этом учитывается, что LLM-оценщик и классификатор работают на разных корпусах (англоязычный RAGBench и русскоязычный RAGBench-RU соответственно), поэтому сравнение трактуется как сопоставление с опорным ориентиром.

5. **Сервис применения (раздел 7).** Реализовать программный интерфейс на базе Streamlit для применения обученных моделей в режимах одиночной и пакетной обработки данных.

4 Русскоязычный набор данных RAGBench-RU

4.1 Исходный RAGBench и отбор поднаборов

Оригинальный RAGBench [3] включает 12 поднаборов, относящихся к пяти предметным доменам. В рамках настоящей работы для построения русскоязычного аналога был сформирован поднабор из 7 частей исходного корпуса, обеспечивающий одновременно доменное разнообразие и достаточный объем данных для обучения и оценки моделей:

- **общие знания:** `hotpotqa`, `hagrid`, `expertqa`;
- **биомедицина:** `covidqa`;
- **клиентская поддержка:** `delucionqa`;
- **юридические контракты:** `cuad`;
- **финансы:** `finqa`.

Оставшиеся поднаборы (`msmarco`, `pubmedqa`, `emannual`, `techqa`, `tatqa`) в данную версию корпуса не включались. Основными критериями отбора были: (а) сохранение доменного разнообразия; (б) наличие достаточного числа примеров в `train/val/test`-разбиениях; (в) разумный баланс между стоимостью перевода и ожидаемой исследовательской пользой.

Полученный русскоязычный корпус содержит около 31 тыс. примеров, разделенных на обучающую, валидационную и тестовую выборки. Этого объема оказалось достаточно для основной серии экспериментов, описанной в разделе 6.

4.2 Схема одного примера

Каждая запись в RAGBench и RAGBench-RU описывается JSON-подобной структурой, содержащей вопрос (`question`), список документов в виде списков предложений (`documents_sentences`), ответ (`response`), а также разметку в форме списков ключей релевантных и использованных предложений и булева флага `adherence_score`. Пример записи из `delucionqa` в сокращенном виде:

```

{
  "question": "Does the 2019 Jeep Cherokee have a backup camera?",
  "documents_sentences": [
    [{"0a", "Jeep Cherokee..."}, {"0b", "ParkView backup camera..."}]
  ],
  "response": "Yes, the 2019 Jeep Cherokee includes a backup camera...",
  "all_relevant_sentence_keys": ["0b"],
  "all_utilized_sentence_keys": ["0b"],
  "adherence_score": true
}

```

Для модели-оценщика из таких записей формируется последовательность $[q][SEP][\text{документы}][SEP][a]$ (см. раздел 3). Метки `all_relevant_sentence_keys` и `all_utilized_sentence_keys` разворачиваются в бинарные маски на токенах документов. Поле `adherence_score` является булевой оценкой всего ответа; в обучающем конвейере оно технически повторяется для всех токенов ответа, чтобы использовать единую схему вычисления функции потерь. Поэтому метка `adherence` не интерпретируется как локализация неподтверждённых фрагментов внутри ответа.

4.3 Схема перевода корпуса

Ядром построения RAGBench-RU является асинхронный конвейер машинного перевода на основе модели Qwen2.5-72B-Instruct [13]. Главной задачей при его проектировании было не только перевести текстовые поля примеров, но и сохранить их внутреннюю структуру, от которой зависит возможность переноса исходной TRACe-разметки без повторной аннотации.

Основные принципы организации конвейера были следующими.

1. **Раздельная обработка полей примера.** Вопрос, ответ и предложения документов переводятся независимыми запросами к модели. При этом предложения документов передаются по одному, в исходном порядке, что позволяет сопоставить каждое переведенное предложение с его исходным ключом (0a, 0b, ...). Такая декомпозиция снижает риск того, что модель объединит, разделит или поменяет местами предложения внутри документа.
2. **Единая инструкция переводчику.** Все запросы выполняются с одной системной инструкцией, в которой модель позиционируется как профессиональный пе-

реводчик с английского на русский и от нее требуется выдавать только перевод, без рассуждений, пояснений и служебных пометок.

3. **Асинхронная пакетная обработка.** Запросы группируются в пакеты и отправляются модели через асинхронный клиент. Размер пакета подбирался эмпирически: слишком маленькие пакеты снижали скорость перевода, а слишком большие чаще приводили к ошибкам при обращении к внешнему сервису применения модели. Для отдельных запросов предусмотрены повторные попытки на случай временных ошибок соединения или ответа сервиса.
4. **Поэтапное сохранение результатов.** Перевод каждого поднабора и разбиения ведется блоками, а результаты по мере готовности дописываются в построчный файл промежуточного состояния. Это позволяет возобновлять перевод с места обрыва без повторной обработки уже переведенных примеров.
5. **Перенос разметки без модификации.** При сборке итогового примера переведенные тексты записываются в новые поля (`question_ru`, `response_ru`, `documents_sentences_ru`), тогда как ключи предложений и поля разметки исходного RAGBench копируются без изменений. Это обеспечивает техническую возможность переноса меток `all_relevant_sentence_keys` и `all_utilized_sentence_keys` на русскоязычную версию корпуса.

Перед запуском перевода из корпуса исключаются записи с пустыми списками предложений, а также примеры, превышающие ограничение по длине входа для модели-переводчика. Содержательная инспекция качества перевода и ограничения, связанные с отсутствием полной ручной постредакции, рассматриваются в разделе 4.4.

4.4 Контроль качества перевода и перенос разметки

Контроль качества построенного корпуса проводился в трех взаимодополняющих формах. Он был направлен прежде всего на проверку структуры данных: сохранности идентификаторов примеров и ключей предложений, согласованности полей и отсутствия явных сбоев перевода, а не на полноценную лингвистическую верификацию независимыми аннотаторами.

1. Автоматические структурные проверки. Из исходного корпуса предварительно исключались записи с пустыми списками предложений, а также примеры, не помещающиеся в контекст переводящей модели. Сама процедура перевода построена так, что переводится только текстовое содержимое предложений, тогда как их идентификаторы и иерархия документов переносятся из исходного RAGBench без изменений. Это позволяет переносить оригинальную разметку `all_relevant_sentence_keys` и `all_utilized_sentence_keys` в RAGBench-RU без повторной автоматической аннотации и связанных с ней искажений целевых меток.

2. Выборочный визуальный просмотр. Отсмотрены случайные примеры из каждого поднабора на предмет явных сбоев перевода: обрывов предложений, смещения русского и английского текста, потери или дублирования предложений, а также нарушения ключей `a/b/c/....`. Результаты такого просмотра использовались итеративно для уточнения системного промпта и общей инструкции перевода. При этом формальная разметка ошибок и количественная оценка адекватности перевода на данном этапе не проводились.

3. Сохранение структуры разметки. Русскоязычная разметка не строится заново, а переносится из исходного RAGBench по сохранённым ключам предложений `a/b/c/....`. Поскольку русскоязычные примеры используют те же ключи предложений, что и английские оригиналы, разметка `all_relevant_sentence_keys` и `all_utilized_sentence_keys` переносится без изменения. Такой прием допустим только при условии сохранения структуры примера и числа предложений, поэтому именно структурные проверки играют в конвейере принципиальную роль.

Следует явно отметить, чего не было сделано в рамках данной работы. Полноценная ручная проверка перевода носителями языка, независимый перевод контрольной выборки, подсчет межаннотаторского согласия и количественная оценка адекватности перевода не проводились. Поэтому RAGBench-RU следует рассматривать как воспроизводимый рабочий бенчмарк для обучения и сравнения моделей, но не как полностью вручную валидированный эталон перевода. Это ограничение далее учитывается при интерпретации результатов.

4.5 Статистика корпуса

Итоговые размеры обучающей, валидационной и тестовой выборок по 7 поднаборам представлены в таблице 2. Распределение по доменам остается достаточно разнообразным: поднаборы общих знаний (`hotpotqa`, `hagrid`, `expertqa`) составляют около 30% корпуса, а крупнейшая специализированная часть приходится на финансовый поднабор `finqa`.

Таблица 2: Размеры обучающей, валидационной и тестовой выборок RAGBench-RU по поднаборам

поднабор	обучение	валидация	тест	домен
hotpotqa	1880	424	390	общие знания
hagrid	2890	322	1322	общие знания
expertqa	1490	196	188	общие знания
covidqa	1250	267	246	биомедицина
delucionqa	1460	182	184	клиентская поддержка
cuad	749	269	305	юриспруденция
finqa	12500	1770	2290	финансы
итого	22 219	3 430	4 925	-

Дополнительные сведения о составе корпуса доступны в карточке Hugging Face: для каждого поднабора указаны размеры обучающей, валидационной и тестовой выборок, типы полей и базовые диапазоны длин текстовых колонок. Эти сведения используются как описательная статистика корпуса, но не являются замером длины полного входа после токенизации конкретной базовой моделью. Поэтому эксперимент с длиной контекста в разделе 6 трактуется не как следствие предварительно измеренного распределения длин в токенах, а как эмпирическая проверка чувствительности обучаемого оценщика к доступному объёму контекста.

4.6 Публикация

Итоговый корпус опубликован на платформе Hugging Face Datasets под именем `CMCenjoyer/ragbench-ru`¹. Публикация включает все 7 поднаборов с разбиением на обучающую, валидационную и тестовую выборки. Лицензия корпуса совпадает с лицензией исходного RAGBench.

В файле `README` указаны источник данных, способ перевода и ключевые ограничения корпуса, в том числе отсутствие полной ручной постредакции. Такая открытая фиксация условий построения корпуса важна для корректной интерпретации результатов и для дальнейшего использования RAGBench-RU другими исследователями.

¹<https://huggingface.co/datasets/CMCenjoyer/ragbench-ru>

5 Архитектура модели-оценщика

5.1 Общая схема

Модель-оценщик представляет собой базовую трансформерную модель с тремя выходными классификационными блоками на уровне токенов, соответствующими целям TRACe: relevance, utilization, adherence. На вход модели поступает объединённая последовательность $[q][SEP][d_1, \dots, d_{|D|}][SEP][a]$, токенизированная единым токенизатором базовой модели. На всех слоях базовой модели используется стандартный механизм самовнимания; дополнительные зависимости между токенами в выходных классификационных блоках не моделируются.

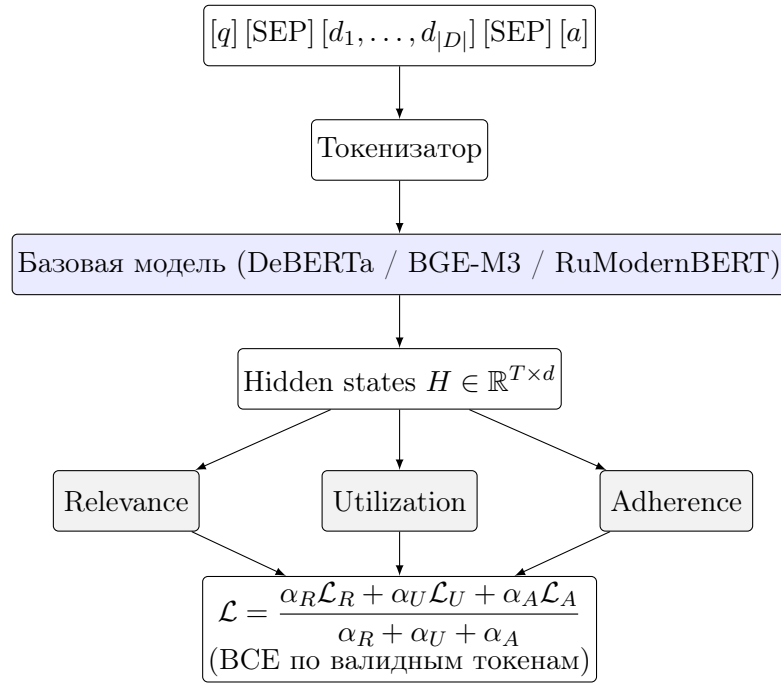


Рис. 2: Общая схема токенов-уровневого оценщика TRACe.

5.2 Архитектуры выходных классификационных блоков

Простая классификационная архитектура (Simple). Воспроизводит архитектуру из оригинальной статьи RAGBench [3]: после энкодера применяется общий слой Dropout [14] и три независимых линейных слоя $\mathbb{R}^d \rightarrow \mathbb{R}^1$, по одному для каждого целе-

вого признака. Формально, для скрытого состояния h_t токена t :

$$\ell_t^{(k)} = W^{(k)}h_t + b^{(k)}, \quad k \in \{R, U, A\},$$

где $W^{(k)} \in \mathbb{R}^{1 \times d}$. Число параметров в выходных блоках составляет $3(d + 1)$ - порядка 2.3 тыс. для базовой модели с $d = 768$ и 4.6 тыс. для $d = 1024$.

Сложная классификационная архитектура (Complex). Архитектура с общим MLP-преобразованием [15] и индивидуальными MLP-блоками с функцией активации GELU [16]:

$$\begin{aligned} x_t &= \text{LN}(\text{Dropout}(\text{GELU}(W_1 h_t + b_1))) \in \mathbb{R}^{d_s}, \\ \tilde{h}_t &= \text{LN}(W_2 x_t + h_t), \\ \ell_t^{(k)} &= V_2^{(k)}(\text{Dropout}(\text{GELU}(\text{LN}(V_1^{(k)} \tilde{h}_t)))). \end{aligned}$$

Здесь d_s совпадает с d (`shared_mult = 1`), а внутренняя размерность индивидуального блока равна $d/2$ (`head_mult = 0.5`). Число параметров в выходных блоках возрастает примерно на два порядка: ~ 2.5 М для $d = 768$.

Сравнение двух вариантов вынесено в раздел 6.4. Здесь важно лишь отметить, что обе архитектуры рассматриваются как конкурирующие гипотезы, а их практическая устойчивость проверяется уже на этапе экспериментов.

5.3 Функция потерь

Функция потерь задаётся как нормированная взвешенная сумма бинарных кросс-энтропий по валидным токенам:

$$\mathcal{L}(\theta) = \frac{\alpha_R \mathcal{L}_R + \alpha_U \mathcal{L}_U + \alpha_A \mathcal{L}_A}{\alpha_R + \alpha_U + \alpha_A},$$

где для $k \in \{R, U\}$ суммирование идёт по маске документов $m_t^{(C)}$, а для $k = A$ - по маске ответа $m_t^{(A)}$. Рассматриваются две схемы весов:

- $(\alpha_R, \alpha_U, \alpha_A) = (1, 1, 1)$ - базовая;
- $(\alpha_R, \alpha_U, \alpha_A) = (3, 3, 1)$ - гипотеза: adherence семантически более простая задача (булев флаг по предложению, а не расстановка токеновых меток), и её пониженный вес позволит сильнее сконцентрировать обучение на релевантности и использовании.

Далее в разделе 6.3 эта гипотеза проверяется экспериментально; как будет показано, различие между двумя схемами весов оказывается небольшим.

5.4 Подбор порогов

После обучения модель выдаёт значения $\ell_t^{(k)}$ для каждой целевой метки $k \in \{R, U, A\}$. Эти значения преобразуются в бинарные предсказания с помощью сигмоидной функции и порога: $\hat{y}_t^{(k)} = \mathbb{I}[\sigma(\ell_t^{(k)}) \geq \tau_k]$.

Пороги τ_R, τ_U, τ_A подбираются независимо для каждой целевой метки на валидационной выборке. Реализация в `model_utils/metrics.py` сводится к одному проходу по валидации для сбора вероятностей и перебору сетки кандидатов $\tau \in \{0.1, 0.2, \dots, 0.7\}$. Для каждого кандидата вычисляется F1-мера на уровне токенов по маске соответствующего целевой метки, после чего выбирается порог, максимизирующий данную метрику.

Как показывают результаты раздела 6.6, оптимальные пороги заметно отклоняются от 0.5 и зависят как от модели, так и от целевой метки. Для `relevance` и `utilization` медианный порог на русскоязычных конфигурациях лежит около 0.4, с разбросом по диапазону $[0.2, 0.5]$. Для `adherence` оптимум часто приходится на нижнюю границу сетки ($\tau_A \approx 0.1$), что объясняется высокой долей положительных примеров: модель уже на низком пороге выдаёт почти все положительные предсказания.

5.5 Выбор базовой модели

В основной сетке экспериментов используются три базовые модели, охватывающие основные классы архитектур, применимых к русскоязычной задаче:

- **DeBERTa-v3-large** (MoritzLaurer/DeBERTa-v3-large-mnli-fever-anli-ling-wanli) - англоязычная базовая конфигурация из оригинальной статьи RAGBench, преобучена на NLI-задачах. Максимальная длина контекста: 512. Используется как контрольный эксперимент и для сравнения RU/EN.
- **BGE-M3** (BAAI/bge-m3 [17]) - мультиязычная модель, обученная для поиска; архитектурно - XLM-RoBERTa-large с поддержкой длинного контекста (до 8192 через масштабирование позиционных кодировок).

- **RuModernBERT** (deepvk/RuModernBERT-base [18]) - русскоязычная модель ModernBERT, обученная на русскоязычном корпусе; поддерживает обработку последовательностей длиной до 8192 токенов за счёт использования вращательных позиционных эмбеддингов (rotary position embeddings [19]).

Дополнительно на предварительном этапе рассматривалась модель **ruRoPEBert-e5-base-2k** (Tochka-AI/ruRoPEBert-e5-base-2k) - русскоязычный BERT с RoPE и контекстом до 2048 токенов, предобученный на задачах поиска. Результаты предварительного эксперимента (раздел 5.6) показали, что эта модель существенно уступает альтернативам в задаче токен-уровневой классификации, поэтому она была исключена из основной сетки.

Мотивация выбора: (а) DeBERTa дублирует базовую конфигурацию статьи и даёт верхнюю оценку качества на английском; (б) BGE-M3 - один из популярных сильных публично доступных мультязычных энкодеров с длинным контекстом; (в) RuModernBERT - современная русскоязычная модель с rotary-позиционными кодировками и поддержкой длинного контекста до 8192 токенов. Таким образом, сетка охватывает три ортогональных оси: язык предобучения (EN / мультязычная / RU), размер модели и схему позиционных кодировок (абсолютные vs rotary).

5.6 Предварительный эксперимент с ruRoPEBert

Модель ruRoPEBert-e5-base-2k запускалась в идентичных с остальными базовыми моделями условиях (RU, Simple, веса функции потерь 1:1:1) на длинах контекста 512, 1024 и 2048 как предварительная проверка перед включением в основную сетку. На всех трёх длинах F1 по relevance и utilization оставался практически нулевым; adherence при этом работал на уровне ≈ 0.85 , однако само по себе такое значение не свидетельствует о хорошем качестве модели, поскольку доля положительных примеров по adherence высока, и константный предсказатель «всегда true» достигает сопоставимого F1 (см. обсуждение в разделе 6.4). По этой причине ruRoPEBert был исключён из основной сетки, чтобы сосредоточить вычислительный бюджет на более перспективных конфигурациях.

Возможное объяснение этого результата состоит в том, что предобучение ruRoPEBert по схеме e5 ориентировано прежде всего на задачу поиска (контра-

стивное обучение эмбедингов предложений), что плохо согласуется с классификацией фрагментов релевантных и использованных предложений на уровне токенов. В отличие от DeBERTa (NLI-предобучение, более близкое к задаче следования) и RuModernBERT (универсальная MLM-модель), ruRoPEBert, по-видимому, формирует представления, лучше приспособленные к агрегированному сопоставлению, чем к позиционному разметочному сигналу. Помимо этого, нельзя полностью исключать и инженерные причины (например, особенности агрегации скрытых состояний или взаимодействия модели с маской на длинном входе); проверка таких гипотез потребовала бы отдельных запусков, и в условиях ограниченного вычислительного бюджета было принято решение перераспределить ресурсы в пользу более перспективных базовых моделей. Поэтому приведённая интерпретация остаётся рабочей гипотезой, хотя и хорошо согласуется с наблюдаемыми результатами.

Таблица 3: Результаты предварительного эксперимента на длине контекста 512 (RU, Simple, веса 1:1:1). ruRoPEBert выдаёт нулевой F1 на relevance и utilization; значение adherence ≈ 0.85 объясняется более легким предсказанием, так как эта целевая метка всегда в конце.

Модель	$F_1^{(R)}$	$F_1^{(U)}$	$F_1^{(A)}$
DeBERTa-v3-large	0.73	0.75	0.897
RuModernBERT-base	0.71	0.74	0.869
ruRoPEBert-e5-base-2k	0.00	0.00	0.852

Зависимость F1 от длины контекста с разбиением по целевым меткам для всех трёх моделей приведена на рис. 3: у ruRoPEBert кривые для relevance и utilization остаются на нуле во всём проверенном диапазоне длин, тогда как у DeBERTa и RuModernBERT значения лежат в рабочем диапазоне 0.7–0.9.

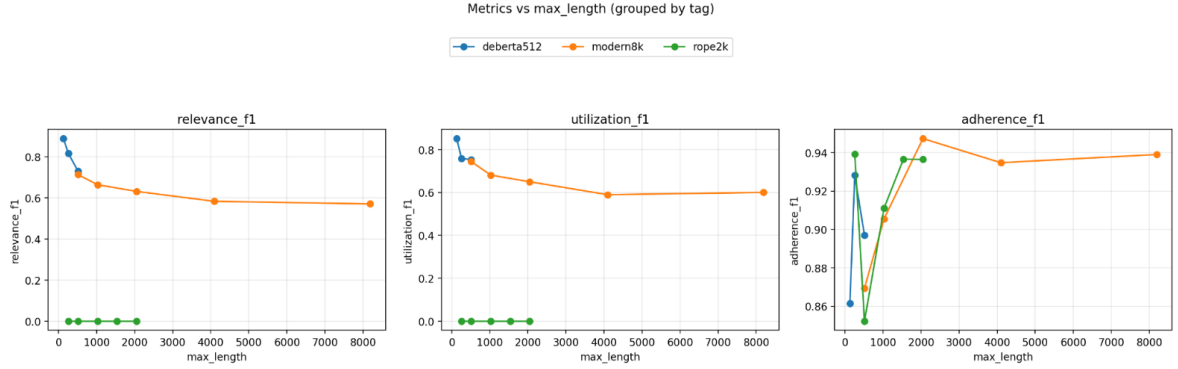


Рис. 3: Зависимость F_1 от длины контекста с разбиением по целевым меткам (relevance, utilization, adherence) для DeBERTa-v3-large, RuModernBERT-base и ruRoPEBert-e5-base-2k (RU, Simple, веса 1:1:1). У ruRoPEBert линии для relevance и utilization лежат на нуле при всех длинах.

5.7 Детали обучения

Все эксперименты выполнялись в единой реализации (модуль `model_utils/`, конвейер `runner.py + train_loop.py`):

- **Оптимизатор:** AdamW с отдельными группами параметров для базовой модели ($\text{lr} = 2 \times 10^{-5}$) и выходных блоков ($\text{lr} = 3 \times 10^{-4}$), `weight_decay` = 0.001.
- **Warmup:** линейный разогрев на 10% шагов, затем линейное затухание до нуля.
- **Батч:** 64 для обучения, 32 для валидации/теста. Для длин ≥ 2048 батч уменьшался до 2-4 с накоплением градиентов.
- **Точность:** смешанная точность FP16 (torch AMP); `grad_clip` = 1.0.
- **Число эпох:** 4 для всех конфигураций; при этом сохранялась лучшая по качеству на валидационной выборке модель, что позволяло ограничить переобучение.

Подбор гиперпараметров выполнялся только на оси «архитектура $\times$ длина $\times$ веса функции потерь»: остальные параметры (скорость обучения, разогрев скорости обучения, Dropout, регуляризация весов) зафиксированы. Такой дизайн позволяет интерпре-

тировать вариации F1 как эффект изучаемых факторов, а не случайный шум гиперпараметрического поиска.

6 Экспериментальная часть

В данном разделе изложены основные эмпирические результаты работы. Его задача состоит не только в том, чтобы перечислить лучшие конфигурации, но и в том, чтобы показать, какие архитектурные и методологические решения оказываются устойчивыми при переносе RAGBench/TRACe на русский язык.

Общая сетка экспериментов имеет следующий вид:

- 3 базовые модели (DeBERTa-v3-large, BGE-M3, RuModernBERT-base); модель ruRoPEBert-e5-base-2k проверялась на предварительном этапе и исключена из основной сетки - см. раздел 5.6;
- до 7 длин контекста (128, 256, 512, 1024, 2048, 4096, 8192) - в зависимости от ограничений базовой модели;
- 2 схемы весов функции потерь (1:1:1, 3:3:1);
- 2 языка разметки (RU, EN);
- 2 архитектуры выходного классификационного блока (Simple, Complex).

В сумме получается около 80 обученных конфигураций. Все результаты сохранены в виде CSV в `analysis_output/tables/`. Ниже сначала приводится краткая сводка, а затем подробно разбираются отдельные факторы.

6.1 Краткая сводка по экспериментам

Таблица 4 агрегирует средний Test F1 по пяти основным сериям экспериментов. Уже на этом уровне видны два устойчивых наблюдения:

1. Русскоязычные конфигурации (RU Simple 1:1:1, RU Simple 3:3:1) систематически опережают англоязычные (EN Simple) как по среднему, так и по максимальному значению $\bar{F}_1$ - интерпретация в разделе 6.5.
2. сложная классификационная архитектура на русском (RU Complex 3:3:1) даёт заметно более низкий средний F1 (0.574 против 0.826–0.829 у Simple), хотя её *максимум* сопоставим с Simple - подробнее в разделе 6.4.

Таблица 4: Средний и максимальный Test F1 по экспериментам (агрегация по всем моделям и длинам)

эксперимент	$\bar{F}_1$ (mean)	$\bar{F}_1$ (max)	$F_1^{(R)}$ max	$F_1^{(U)}$ max
RU Simple 1:1:1	0.8293	0.8539	0.8837	0.8468
RU Simple 3:3:1	0.8256	0.8578	0.8869	0.8467
EN Simple 1:1:1	0.6091	0.7244	0.6399	0.6267
EN Simple 3:3:1	0.5727	0.7467	0.6589	0.6590
RU Complex 3:3:1	0.5743	0.8363	0.8869	0.7519

Топ-10 конфигураций по $\bar{F}_1$ приведён в рейтинговой таблице 5: все места занимают RU Simple-конфигурации, лучшая - **ModernBERT-base**, длина **2048**, **Simple 3:3:1**, $\bar{F}_1 = 0.8578$.

Таблица 5: Топ-10 конфигураций по среднему Test F1 (по всем экспериментам)

эксперимент	модель	длина	$\bar{F}_1$	$F_1^{(R)}$	$F_1^{(U)}$	$F_1^{(A)}$
RU Simple 3:3:1	ModernBERT	2048	0.8578	0.7978	0.8406	0.9349
RU Simple 1:1:1	BGE-M3	2048	0.8539	0.7956	0.8308	0.9352
RU Simple 1:1:1	ModernBERT	2048	0.8535	0.7968	0.8328	0.9309
RU Simple 3:3:1	BGE-M3	2048	0.8526	0.7962	0.8249	0.9368
RU Simple 3:3:1	DeBERTa-v3	512	0.8477	0.7829	0.8467	0.9135
RU Simple 1:1:1	BGE-M3	512	0.8472	0.7938	0.8161	0.9318
RU Simple 1:1:1	ModernBERT	1024	0.8414	0.7769	0.8264	0.9209
RU Simple 3:3:1	ModernBERT	4096	0.8412	0.7682	0.8188	0.9366
RU Simple 3:3:1	ModernBERT	1024	0.8412	0.7757	0.8234	0.9245
RU Simple 1:1:1	DeBERTa-v3	512	0.8406	0.7732	0.8468	0.9018

6.2 Сетка моделей и длин контекста

На рис. 4 показана зависимость $\bar{F}_1$ от длины контекста для трёх базовых моделей на русском (Simple, веса 3:3:1). Основные выводы:

- **ModernBERT-base** демонстрирует самое устойчивое поведение: плато

$\bar{F}_1 \approx 0.84\text{--}0.86$ в диапазоне 1024–4096, с небольшим падением на 8192 (до ~ 0.83).

- **BGE-M3** близок к ModernBERT по максимуму ($\bar{F}_1 = 0.853$ при длине 2048), но на длинах ≤ 512 работает слабее из-за «размытия» релевантных токенов после масштабного многоязычного предобучения.
- **DeBERTa-v3-large** - наиболее сильная базовая модель для коротких последовательностей ($\bar{F}_1 = 0.848$ при длине 512); однако ограничение на длину входа в 512 токенов приводит к потере части контекста на поднаборах с длинными документами (cuad, finqa).

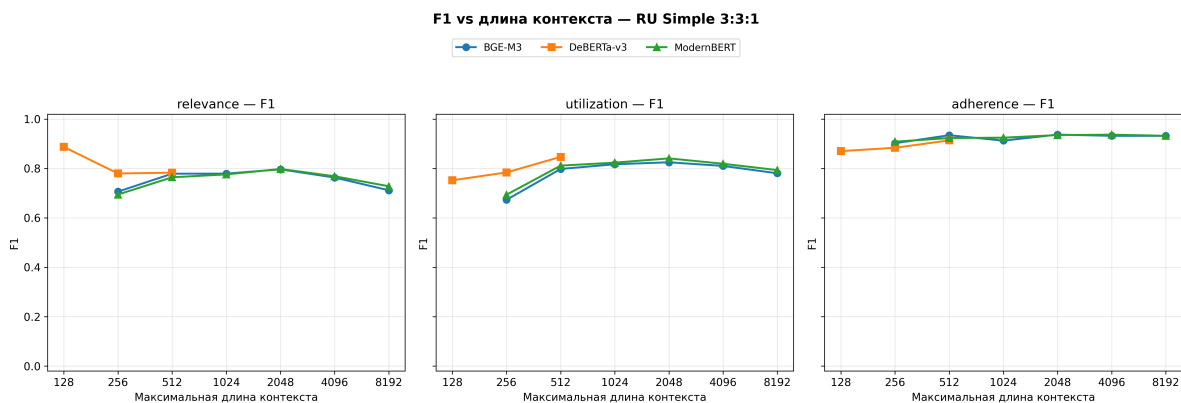


Рис. 4: Зависимость среднего Test F1 от длины контекста (RU, Simple, веса 3:3:1).

Лучшие конфигурации на каждой базовой модели (для RU Simple 3:3:1) отражены в рис. 5: ModernBERT и BGE-M3 устойчиво лидируют, а DeBERTa на своём «родном» лимите 512 почти догоняет лидеров.

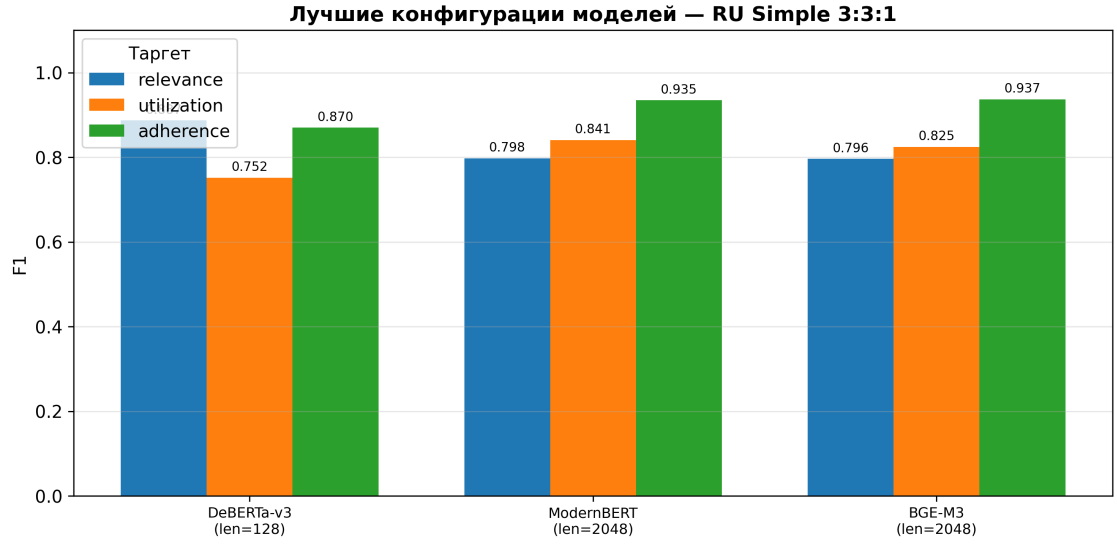


Рис. 5: Максимальный $\bar{F}_1$ по моделям (RU Simple 3:3:1); над столбцом - оптимальная длина контекста.

Оптимальная длина. На подавляющем большинстве поднаборов максимум достигается при длине 2048: при 512 обрезаются длинные документы *cuad/finqa*, а при 4096-8192 модель начинает «шуметь» - доля положительных токенов в большой последовательности мала, и без дополнительного взвешивания классов сигнал размывается.

6.3 Анализ влияния весов функции потерь

Схема 3:3:1 была сформулирована с гипотезой, что adherence - более простая задача и не требует большого веса. Эмпирически (рис. 6) разница между 1:1:1 и 3:3:1 на русском не превосходит $|\Delta F_1| \approx 0.02$ ни для одной конфигурации; средний $|\Delta F_1|$ составляет ≈ 0.004 . Аналогичная картина на английском (*ablation_weights_EN_Simple_111_vs_EN_Simple_331.png*).

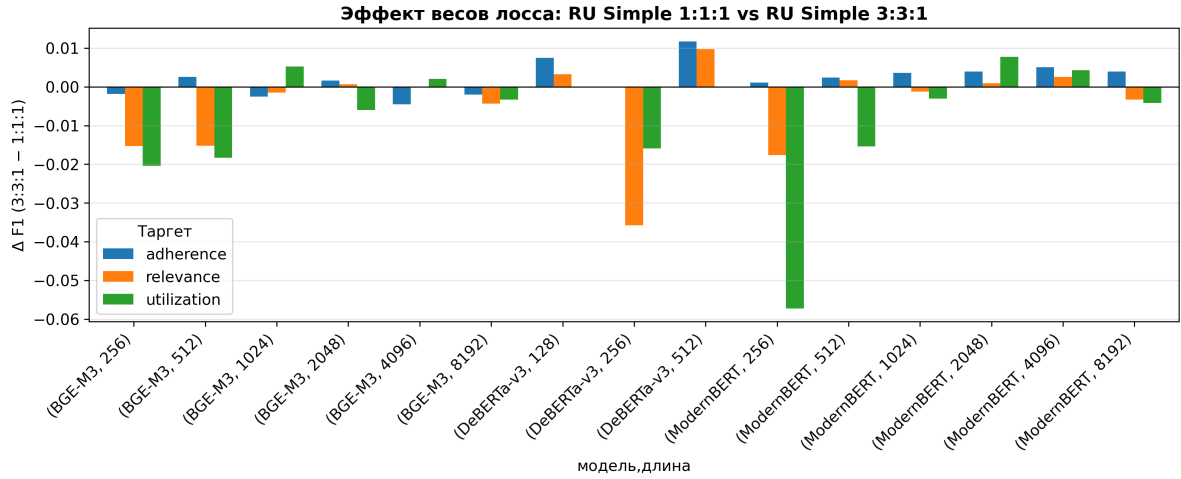


Рис. 6: Абляция весов функции потерь на русском (Simple): $\Delta F_1(3:3:1 - 1:1:1)$ по моделям и длинам. Полоска в поле = безразличный к весам случай.

Детальная таблица ΔF_1 (CSV `ablation_weights_RU_Simple_111_vs_RU_Simple_331.csv`) показывает, что знак Δ непостоянен внутри одной модели. Итоговый вывод: *гипотеза о необходимости 3:3:1 не подтверждается*, дисбаланс весов не даёт систематического эффекта при фиксированном остальном наборе гиперпараметров.

6.4 Сравнение Simple и Complex

Сравнение классификационных архитектур Simple и Complex на русском (веса 3:3:1) представлено в таблице 6 и на рис. 7.

Таблица 6: Simple vs Complex head на RU (веса 3:3:1): Δ F1 по таргетам

модель	длина	таргет	F1 (Simple)	F1 (Complex)	Δ (Complex - Simple)
BGE-M3	1024	relevance	0.7789	0.4294	-0.3495
BGE-M3	2048	relevance	0.7962	0.3207	-0.4755
BGE-M3	4096	relevance	0.7630	0.3226	-0.4404
BGE-M3	512	relevance	0.7786	0.5187	-0.2599
BGE-M3	8192	relevance	0.7116	0.3076	-0.4040
DeBERTa-v3	128	relevance	0.8869	0.8869	0.0000
DeBERTa-v3	256	relevance	0.7794	0.6842	-0.0953
DeBERTa-v3	512	relevance	0.7829	0.5649	-0.2180
ModernBERT	1024	relevance	0.7757	0.5680	-0.2077
ModernBERT	2048	relevance	0.7978	0.5668	-0.2310
ModernBERT	4096	relevance	0.7682	0.0000	-0.7682
ModernBERT	512	relevance	0.7640	0.5569	-0.2071
ModernBERT	8192	relevance	0.7278	0.3087	-0.4191
BGE-M3	1024	utilization	0.8168	0.3601	-0.4567
BGE-M3	2048	utilization	0.8249	0.2711	-0.5538
BGE-M3	4096	utilization	0.8100	0.2613	-0.5486
BGE-M3	512	utilization	0.7978	0.4620	-0.3358
BGE-M3	8192	utilization	0.7803	0.2367	-0.5436
DeBERTa-v3	128	utilization	0.7519	0.7519	0.0000
DeBERTa-v3	256	utilization	0.7837	0.6017	-0.1820
DeBERTa-v3	512	utilization	0.8467	0.4869	-0.3598
ModernBERT	1024	utilization	0.8234	0.5263	-0.2971
ModernBERT	2048	utilization	0.8406	0.5340	-0.3066
ModernBERT	4096	utilization	0.8188	0.0000	-0.8188
ModernBERT	512	utilization	0.8113	0.4962	-0.3150
ModernBERT	8192	utilization	0.7932	0.2378	-0.5554
BGE-M3	1024	adherence	0.9125	0.9114	-0.0011
BGE-M3	2048	adherence	0.9368	0.9305	-0.0063
BGE-M3	4096	adherence	0.9319	0.9280	-0.0038
BGE-M3	512	adherence	0.9343	0.9191	-0.0152
BGE-M3	8192	adherence	0.9317	0.9275	-0.0042
DeBERTa-v3	128	adherence	0.8700	0.8700	0.0000
DeBERTa-v3	256	adherence	0.8835	0.8835	0.0000

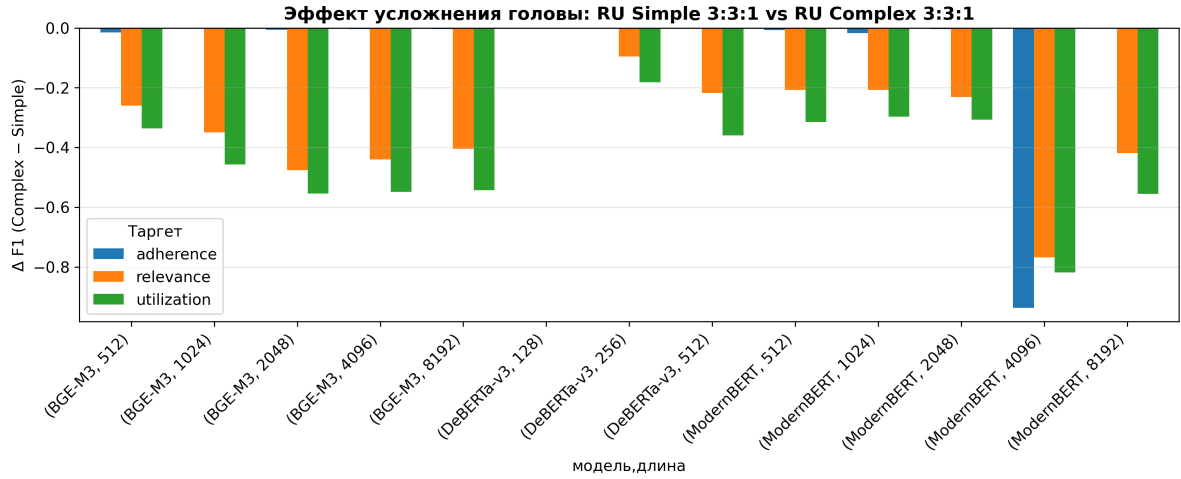


Рис. 7: Simple и Complex: $\Delta F_1(\text{Complex} - \text{Simple})$ по целевым меткам и моделям (RU, веса 3:3:1).

Результат здесь достаточно отчетливый: Complex *стабильно уступает* Simple по relevance и utilization, причем отставание увеличивается с ростом длины контекста. Для BGE-M3 при длине 2048 $\Delta = -0.48$, при 4096 - -0.44 . На ModernBERT при длине 4096 наблюдается деградация до нулевого F1, то есть усложнение выходного блока в этой постановке не только не помогает, но и может приводить к деградации модели.

Для adherence различия между Simple и Complex в большинстве конфигураций малы ($|\Delta| \leq 0.02$); исключение составляют отдельные запуски с полной деградацией модели. Это согласуется с гипотезой о том, что данная целевая метка требует меньшей выразительной мощности выходного классификационного блока.

Интерпретация. На имеющемся объеме обучающей выборки (~ 22 тыс. примеров) и при фиксированном бюджете обучения простая линейная классификационная архитектура, по-видимому, оказывается ближе к разумному компромиссу между выразительностью и устойчивостью. Более сложная архитектура может переобучаться и хуже переноситься на длинные последовательности. В этом смысле полученный результат дополнительно поддерживает исходный архитектурный выбор авторов RAGBench.

6.5 Сравнение RU и EN

Главное отличие RAGBench-RU от оригинального RAGBench состоит в переводе всех текстовых полей через Qwen2.5-72B. На рис. 8 показан ΔF_1 (RU - EN) по моделям и длинам при одинаковых архитектуре и весах.

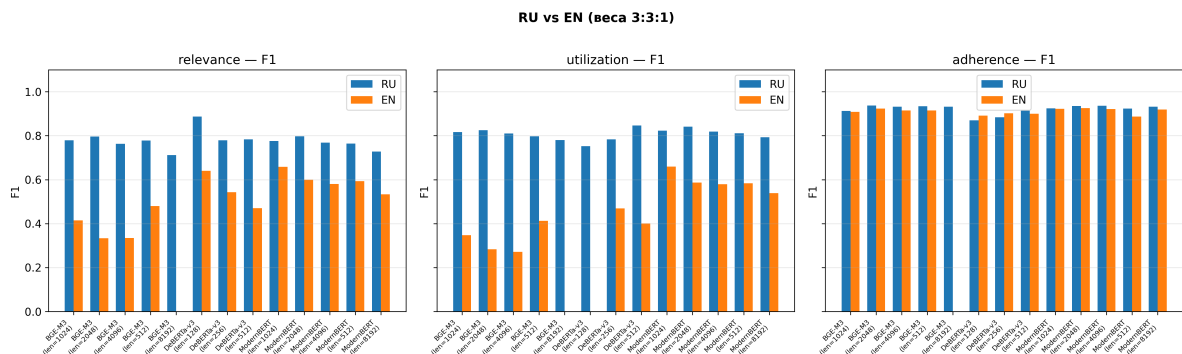


Рис. 8: Сравнение RU и EN (Simple, веса 3:3:1): ΔF_1 (RU - EN) по моделям, длинам и целевым меткам.

В проведённой сетке экспериментов на построенном корпусе русскоязычные конфигурации дают более высокие значения F_1 по relevance и utilization, тогда как на adherence разница мала ($|\Delta| \leq 0.03$). Наблюдаемая разница в ряде конфигураций достигает +0.2–0.4 по F_1 . Это наблюдение устойчиво в пределах проведённых экспериментов, однако его нельзя интерпретировать как чистый эффект языка: русская и английская части различаются не только языком, но также способом построения корпуса (машинный перевод исходных текстов), составом базовых моделей и параметрами токенизации. Ниже приведены две *рабочие гипотезы*, которые могут вносить вклад в наблюдаемое различие:

1. **Согласованность базовой модели и языка.** RuModernBERT и BGE-M3 сильнее работают на русском, чем DeBERTa-v3-large - на английском при 512 токенах; на длинных контекстах русскоязычные базовые модели получают преимущество, которое DeBERTa не может компенсировать из-за ограничения в 512 токенов на длину входа.
2. **Влияние машинного перевода и токенизации.** Перевод меняет поверхностную форму текста, длины последовательностей и распределение токенов; это мо-

жет влиять на удобство обучения модели. В рамках данной работы этот фактор не отделяется от языка и состава базовых моделей.

Перечисленные гипотезы не следует трактовать как доказанные объяснения. Эталонная разметка RAGBench-RU переносится из исходного RAGBench по сохранённым ключам предложений и не пересопоставляется по содержанию или позиции, поэтому утверждать, что перевод улучшает или «очищает» исходную разметку, в рамках данной работы нельзя. По результатам этого подраздела можно утверждать только наличие различия на построенном корпусе; его причина требует отдельного исследования с контролем перевода, токенизации, состава моделей и качества разметки. Сопоставление этих значений с ручной оценкой носителями языка и отделение эффекта языка от эффекта перевода остаются задачами дальнейшей работы.

6.6 Подбор порогов

Рис. 9 показывает распределение оптимальных порогов τ_R, τ_U, τ_A , выбранных на валидации для всех обученных конфигураций.

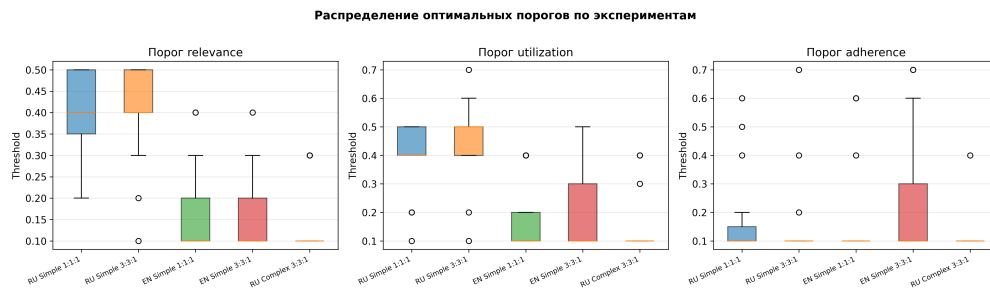


Рис. 9: Распределение оптимальных порогов по целевым меткам (все конфигурации).

Наблюдения:

- Для relevance и utilization медианный порог на RU - около 0.4, с хвостом на 0.2–0.5.
- Для adherence оптимум часто приходится на минимум сетки (0.1), то есть модель уже на низком пороге выдаёт почти все положительные предсказания; это объяснимо высокой долей положительных примеров (большая часть ответов подержана контекстом).

- На EN медианный порог смещён ещё ниже (около 0.1), что соответствует низкому уровню recall из-за шумной разметки.

Фиксированный порог 0.5, принятый по умолчанию в оригинальной статье, даёт $\bar{F}_1$ в среднем на ~ 0.03 – 0.05 ниже оптимального значения. Поэтому подбор порога для каждой целевой метки следует рассматривать как важный компонент итогового конвейера, а не как второстепенную техническую деталь.

6.7 Классификатор vs LLM-оценщик

Отдельный эксперимент посвящён сопоставлению лучшего классификатора с LLM-оценщиком без дообучения на базе Qwen2.5-72B, работающим по схеме, близкой к использованной в оригинальной статье RAGBench. Большая языковая модель получает вопрос, документы и ответ, возвращает списки ключей релевантных и использованных предложений, а также булев флаг `adherence`; далее эти списки ключей сравниваются с эталонными списками на уровне предложений. Значение F1 на уровне токенов для оценщика на базе большой языковой модели не вычислялось.

Методологическое уточнение. Полная формулировка условий сопоставления приведена в постановке задачи (раздел 3, задачи 1 и 4). Здесь достаточно напомнить, что большая языковая модель оценивалась на исходном англоязычном RAGBench на уровне предложений, а классификатор — на русскоязычном RAGBench-RU на уровне токенов. Поэтому сопоставление трактуется как сравнение с опорным ориентиром, а соответствующие клетки в таблице 7 отмечены прочерком.

Таблица 7: Метрики оценщика на базе большой языковой модели Qwen2.5-72B на исходном англоязычном RAGBench: совпадение по предложениям и среднее число пропусков для релевантных и использованных предложений. Русскоязычные данные RAGBench-RU на вход модели не подавались. Эти значения используются как опорный ориентир и не сопоставимы напрямую с метриками классификатора, который оценивается отдельно на RAGBench-RU на уровне токенов.

метод	совпадение по предложениям		среднее число пропусков	
	релевантные	использованные	релевантные	использованные
Qwen2.5-72B (англ. RAGBench)	0.7022	0.8301	1.0547	0.4842

Таблица 7 приводит метрики оценщика на базе большой языковой модели на англоязычном корпусе; качество классификатора, обученного на русскоязычном RAGBench-RU, оценивается отдельно по F1 на уровне токенов (см. раздел 6.1 и обсуждение ниже). Дополнительно рис. 10 показывает распределение точности и полноты по примерам для обоих методов в их собственных шкалах агрегации.

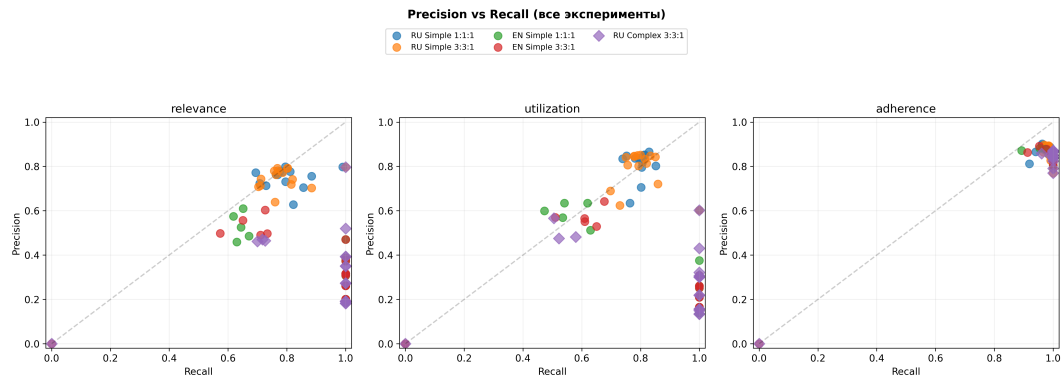


Рис. 10: Точность и полнота по примерам для оценщика на базе большой языковой модели (синие) и лучшего классификатора (оранжевые) на соответствующих тестовых выборках.

Основные выводы.

- Совпадение предложений.** На relevance Qwen достигает 0.70 совпадения с эталоном, то есть заметная доля предложений либо пропускается, либо помечается

лишней; на utilization совпадение выше и составляет 0.83.

2. **Склонность к избыточной разметке.** Метрики `relevance_miss_avg` и `extra` показывают, что LLM имеет тенденцию пометать в среднем около одного лишнего предложения на пример как релевантное. Это согласуется с известной проблемой LLM-оценщиков, отмеченной как в RAGBench [3], так и в AttributionBench [7].
3. **Качество классификатора на уровне токенов.** Основная конфигурация работы (RuModernBERT, RU Simple 3:3:1, длина 2048; см. раздел 6.1) на русскоязычном тестовом корпусе RAGBench-RU даёт $F_1^{(R)} = 0.798$ по `relevance` и $F_1^{(U)} = 0.841$ по `utilization` относительно перенесённой эталонной разметки; значение $F_1^{(A)} \approx 0.93$ по `adherence` в сумме даёт средний $\bar{F}_1 = 0.858$, приведённый в таблице 4. Эти значения приводятся в тексте отдельно от таблицы 7 и не объединяются с ней в одну строку: оценщик на базе большой языковой модели и классификатор работают на разных корпусах (исходный англоязычный RAGBench и русскоязычный RAGBench-RU соответственно) и в разных шкалах агрегации (уровень предложений и уровень токенов). По этой причине вывод о превосходстве одного метода над другим по единой метрике в данной работе не делается, а сопоставление трактуется как сравнение с опорным ориентиром, а не контролируемый эксперимент на одном корпусе. Количественные замеры задержки и стоимости также не проводились; вместе с тем классификатор является локальной компактной моделью и потому используется как более дешёвая прикладная альтернатива оценщику на базе большой языковой модели.

Таким образом, полученные результаты согласуются с ключевой идеей оригинальной статьи RAGBench: компактный дообученный классификатор остаётся конкурентоспособной и существенно более дешёвой альтернативой LLM-оценщику без дообучения. Прямое подтверждение превосходства по единой метрике на уровне токенов требует отдельного эксперимента (запуск LLM-оценщика непосредственно на русскоязычном корпусе RAGBench-RU при той же шкале агрегации, что и у классификатора) и рассматривается как направление дальнейшей работы. Это делает обученные на RAGBench-RU модели перспективными для автономной оценки и массового анализа ответов, где важны стоимость, скорость и воспроизводимость.

6.8 Корреляция со значениями TRACe-метрик

На рис. 4 и в CSV `per_experiment_summary.csv` приведены метрики на уровне токенов; параллельно вычислялись и агрегированные значения TRACe-метрик на уровне примера по схеме, описанной в разделе 2.2. Корреляция Пирсона между предсказанными классификатором значениями и эталонной разметкой RAGBench (GPT-4-аннотация, перенесённая по ключам предложений) составляет $\rho \geq 0.90$ для `relevance` и `utilization` при использовании топ-5 конфигураций RU Simple; для `adherence` $\rho \approx 0.85$. Это указывает на то, что выбранная схема агрегации токеновых предсказаний достаточно хорошо переносит качество классификатора на уровень итоговых TRACe-метрик.

6.9 Применение на внешнем наборе данных SberQuAD-QA

Данный подраздел посвящён проверке переносимости обученного оценщика на внешний русскоязычный набор данных — SberQuAD-QA, содержащий пары «вопрос — фрагмент документа — метка релевантности». Эксперимент проводится *без дообучения* и *без подбора порогов* на внешнем наборе: используются только обученные на RAGBench-RU модели и пороги, зафиксированные на валидации исходного корпуса. Разметка SberQuAD-QA содержит лишь признак релевантности фрагментов и не включает меток использованности и верности, поэтому сравнение проводится только по метке релевантности.

Применяются три обученные конфигурации (DeBERTa-v3-large при длине контекста 512, BGE-M3 при 1024 и ModernBERT при 2048). Пороги классификации берутся из соответствующих прогонов на RAGBench-RU (`result.json`). Примеры, не помещающиеся в максимальную длину предобученной базовой модели, исключаются из подсчёта метрик; итоговые значения F_1 , ROC-AUC и доли верных классификаций приведены по обработанным примерам, а доля пропущенных вынесена в отдельный столбец таблицы 8.

Таблица 8: Применение обученных оценщиков на внешнем русскоязычном датасете SberQuAD-QA без дообучения и без подбора порогов. Сравнение проводится только по релевантности, поскольку разметка SberQuAD-QA не содержит меток использованности и верности. Пороги классификации взяты из соответствующих прогонов на RAGBench-RU. Доля пропущенных - доля примеров, не помещающихся в максимальную длину.

модель	длина контекста	F_1	ROC-AUC	доля верных классификаций	пропущено, %
DeBERTa-v3-large	512	0.577	0.9126	0.694	10.49
BGE-M3	1024	0.668	0.9201	0.815	0.03
ModernBERT	2048	0.757	0.9275	0.883	0.01

Лучший результат показывает ModernBERT: $F_1 = 0.757$, ROC-AUC = 0.928, доля верных классификаций 0.883. Снижение значений относительно показателей на RAGBench-RU ожидаемо и объясняется переносом на другой источник данных и несколько иную схему разметки. У DeBERTa-v3-large заметна доля пропущенных примеров (около 10.5%) из-за ограничения длины контекста в 512 токенов, что снижает покрытие внешнего набора и ограничивает практическую применимость этой конфигурации. Различимость классов сохраняется у всех трёх моделей: значения ROC-AUC лежат в диапазоне 0.91–0.93, что видно на рис. 11.

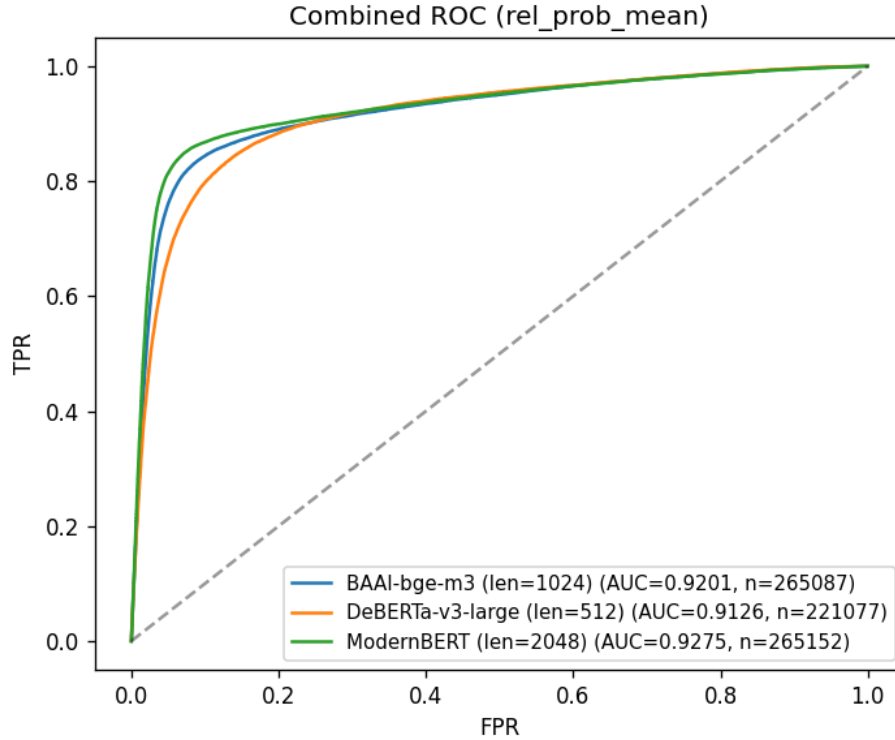


Рис. 11: ROC-кривые трёх обученных оценщиков на внешнем наборе данных SberQuAD-QA по метке релевантности (без дообучения, пороги перенесены из прогонов на RAGBench-RU).

В целом эксперимент показывает, что лучший перенос среди трёх рассмотренных конфигураций демонстрирует ModernBERT: он опережает DeBERTa-v3-large и BGE-M3 по всем трём метрикам (см. таблицу 8). Снижение F_1 относительно показателей на RAGBench-RU ожидаемо и объясняется сменой источника данных и иной схемой разметки. При этом ROC-AUC всех трёх моделей лежит в диапазоне 0,91–0,93, то есть различимость классов при переносе сохраняется и ранжирование примеров по предсказанной оценке остаётся информативным. Тем самым результат подтверждает применимость подхода к внешним русскоязычным задачам определения релевантности, однако не заменяет полноценную TRACe-разметку, поскольку SberQuAD-QA не содержит меток использованности и верности.

6.10 Выводы по экспериментальной главе

Лучшая конфигурация в проведённой сетке - ModernBERT-base при длине контекста 2048 с простой классификационной архитектурой и весами 3:3:1; она достигает $\bar{F}_1 = 0,8578$. Длина контекста около 2048 токенов оказывается наиболее устойчивой: меньшие длины обрезают длинные документы, а большие приводят к размыванию сигнала. Изменение схемы весов функции потерь с 1:1:1 на 3:3:1 не даёт устойчивого практического преимущества, а простая классификационная архитектура стабильно превосходит сложную, особенно на длинных последовательностях. Различие между русскоязычными и англоязычными конфигурациями наблюдается, но не сводится к чистому эффекту языка - оно смешано с влиянием машинного перевода, токенизации и состава базовых моделей. Сопоставление с LLM-оценщиком на базе Qwen2.5-72B играет роль опорного ориентира, а не прямого контролируемого эксперимента на одном корпусе и в одной шкале агрегации. Наконец, эксперимент на внешнем наборе данных SberQuAD-QA подтверждает переносимость подхода по метке релевантности: ранжирование примеров и различимость классов сохраняются, а лучший перенос показывает та же конфигурация на основе ModernBERT.

7 Интерактивный сервис оценки

7.1 Назначение

Помимо опубликованного набора данных и обученных моделей, практическая ценность работы включает интерактивный сервис, позволяющий применять разработанные оценщики к новым RAG-ответам без необходимости обращаться к обучающему конвейеру напрямую. Сервис реализован на Streamlit (модуль `new_streamlit.py`) и запускается командой `streamlit run new_streamlit.py`. Применение модели может выполняться как на CPU, так и на одном GPU, без специальной дополнительной настройки.

Роль сервиса в контексте диссертации прикладная: он не является центральным научным результатом работы, но демонстрирует, что обученные модели можно использовать не только в экспериментальном конвейере, но и в режиме анализа новых примеров.

7.2 Архитектура

Логически сервис состоит из трех блоков.

- **Реестр моделей.** В сервисе задан список доступных конфигураций, соответствующих обученным моделям, предназначенным для практического использования. На момент написания работы доступны три базовые конфигурации:
 - DeBERTa-v3-large (len=512) - контрольная базовая модель;
 - ModernBERT (len=1024) - основная русскоязычная конфигурация;
 - BAAI-bge-m3 (len=1024) - многоязычная модель для смешанных корпусов.

Веса хранятся локально и повторно используются средствами Streamlit, что позволяет не перегружать память повторной инициализацией модели при переключении между сессиями.

- **Модуль применения модели.** Внутри сервиса реализован легковесный модуль загрузки сохранённых весов, совместимый с обучающей реализацией модели-оценщика. На вход он принимает тройку «вопрос – контекст – ответ» в формате, близком к структуре RAGBench-RU, а на выходе формирует вероятности и агрегированные значения TRACe-метрик.

- **Пользовательский интерфейс.** Интерфейс поддерживает два основных режима:
 1. *Одиночный режим* - оценка одной тройки (вопрос, контекст, ответ) с визуализацией результата;
 2. *Пакетный режим* - оценка CSV/JSONL-файла с последующей выгрузкой рассчитанных оценок.

7.3 Основные возможности

- **Одиночная оценка примера.** Пользователь может ввести вопрос, контекст и ответ вручную и получить три основных значения TRACe-метрики, а также вероятности на уровне токенов. Такой режим полезен для разбора конкретных ошибок и качественного анализа поведения модели.
- **Пакетная обработка.** Сервис поддерживает загрузку таблиц с заранее подготовленными примерами и возвращает результаты в виде CSV с рассчитанными значениями `relevance_score`, `utilization_score` и `adherence_score`. Это делает сервис удобным для автономного анализа небольших коллекций ответов.
- **Настройка порогов.** Пороги τ_R , τ_U , τ_A можно изменять интерактивно. Такая возможность важна не только с прикладной точки зрения, но и как средство исследования: она позволяет наблюдать, как меняется визуальная разметка при переходе от более консервативного к более чувствительному режиму.
- **Визуализация разметки на уровне токенов.** В одиночном режиме документ и ответ отображаются с цветовой подсветкой, отражающей предсказания релевантности, использованности и верности. Это помогает интерпретировать модель не только через итоговые оценки, но и через локальные решения на уровне текста.
- **Совместимость с предобученными моделями и локально сохранёнными весами.** Сервис загружает базовые модели и токенизаторы через интерфейс Hugging Face Transformers, а дообученные веса TRACe-оценщика считывает из локально сохраненных файлов. Поэтому он может использоваться как интерфейс к

уже полученным экспериментальным результатам при наличии соответствующих локальных артефактов модели и файла с подобранными порогами.

На рисунке 12 приведен пример работы сервиса на одном из примеров набора данных RAGBench-RU. В интерфейсе выбраны модель, пример из набора данных и пороги классификации, после чего сервис рассчитывает значения TRACe-метрик и сопоставляет их с эталонной разметкой. При выбранных порогах оценки релевантности, использованности и верности близки к эталонным значениям. Кроме того, таблица вероятностей на уровне токенов показывает, какие фрагменты текста модель считает наиболее значимыми для каждой метки: более насыщенный цвет ячейки соответствует большей вероятности.

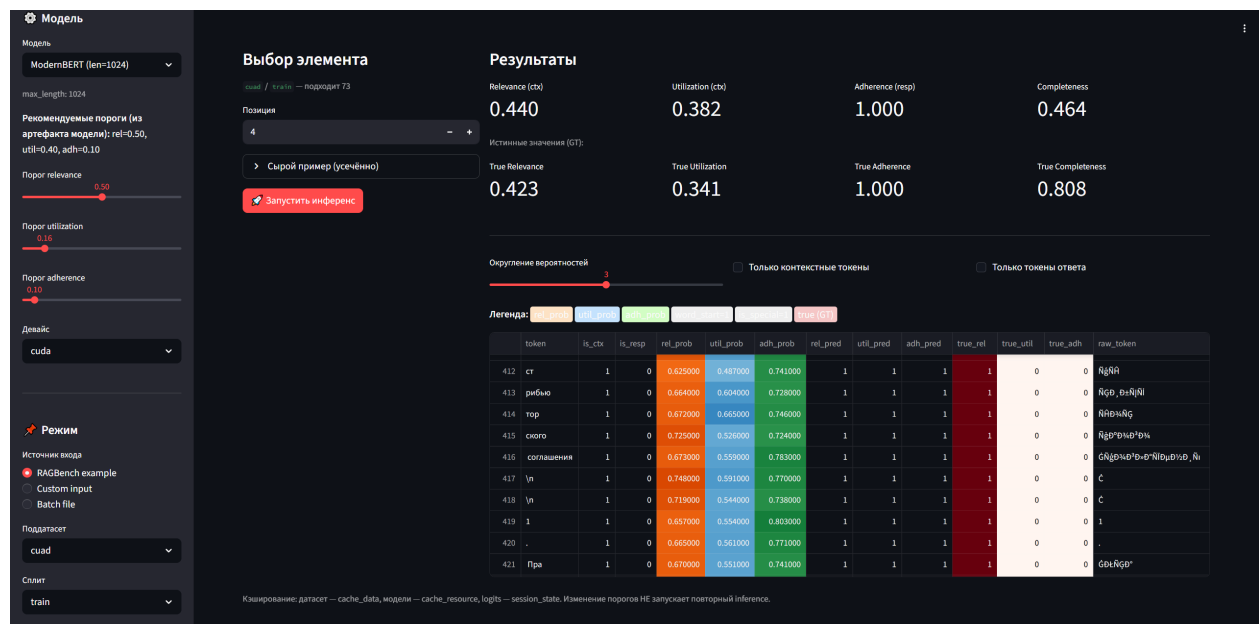


Рис. 12: Пример работы интерактивного сервиса на примере из RAGBench-RU: сравнение рассчитанных TRACe-метрик с эталонной разметкой и визуализация вероятностей на уровне токенов.

7.4 Ограничения

Сервис сознательно не пытается решать все задачи, связанные с жизненным циклом моделей-оценщиков.

- Он ориентирован на одиночную и пакетную автономную оценку, а не на полноценное производственное развертывание.
- Он не содержит встроенного механизма дообучения на пользовательских данных; для таких сценариев следует использовать основной обучающий конвейер проекта.
- В работе не проводились формальные измерения производительности сервиса (задержка, пропускная способность, устойчивость при высокой нагрузке), поэтому приведенная реализация рассматривается как прикладной интерфейс, а не как готовый промышленный компонент.
- Интеграция через REST-API, webhook или CI-контуры в текущей версии не реализована и рассматривается как направление дальнейшего развития.

Таким образом, Streamlit-сервис выступает в работе как удобный мост между исследовательским результатом и его практическим использованием: он показывает, что обученные оценщики можно применять к новым данным уже в текущем виде, но не подменяет собой отдельную задачу сервисного развертывания.

8 Заключение

В настоящей работе предложен и реализован воспроизводимый подход к оценке русскоязычных RAG-систем, основанный на переносе методологии RAGBench/TRACe на русский язык. Работа объединяет несколько связанных результатов: построение русскоязычного бенчмарка, обучение семейства токено-уровневых моделей-оценщиков, анализ их поведения на разных конфигурациях и создание прикладного инструмента для применения обученных моделей. В совокупности эти результаты формируют основу для более систематической оценки русскоязычных RAG-систем как в исследовательских, так и в прикладных сценариях.

8.1 Полученные результаты

1. **Построен русскоязычный бенчмарк.** Опубликован корпус RAGBench-RU², представляющий собой перевод 7 поднаборов исходного RAGBench на Qwen2.5-72B. При построении корпуса реализован конвейер, включающий структурные проверки корректности перевода, сохранности ключей и согласованности разметки.
2. **Воспроизведена схема LLM-оценщика без дообучения.** Сопоставление с Qwen2.5-72B использовалось как опорный ориентир, а не как прямой эксперимент на одном корпусе.
3. **Обучено семейство моделей-оценщиков.** Обучено около 80 конфигураций моделей-оценщиков на разных базовых архитектурах, длинах контекста, схемах весов функции потерь и вариантах выходного классификационного блока. Лучшая конфигурация - **ModernBERT-base**, длина **2048**, **Simple 3:3:1** - достигает $\bar{F}_1 = 0.858$ на объединенном тестовом корпусе.
4. **Проведен сравнительный экспериментальный анализ.** Показано, что на исследованной задаче:

- оптимальная длина контекста для лучших конфигураций находится вблизи

²<https://huggingface.co/datasets/CMCenjoyer/ragbench-ru>

2048 токенов, тогда как слишком короткий контекст приводит к потере информации, а чрезмерно длинный - к размыванию сигнала;

- различие между схемами весов 1:1:1 и 3:3:1 оказывается небольшим и не дает устойчивого практического преимущества;
- простая классификационная архитектура систематически устойчивее сложной, особенно на длинных контекстах;
- на построенном корпусе русскоязычные конфигурации показали более высокие значения F_1 по relevance и utilization, однако интерпретация этого различия ограничена влиянием перевода, токенизации и перенесённой разметки и не может быть сведена к чистому эффекту языка.
- дополнительная проверка на SberQuAD-QA показала, что обученные модели демонстрируют переносимость по метке релевантности на внешнем русскоязычном наборе данных без дообучения и без подбора порогов.

5. Классификатор как практическая альтернатива LLM-оценщику. LLM-оценщик без дообучения на базе Qwen2.5-72B оценивался на исходном англоязычном корпусе RAGBench на уровне предложений (совпадение по предложениям, среднее число пропусков) и демонстрирует заметные расхождения с эталонной разметкой, включая склонность к избыточной разметке релевантных предложений. Обученный классификатор, в свою очередь, достигает высоких значений F_1 на уровне токенов относительно эталонной разметки RAGBench-RU. Прямое сравнение Qwen и классификатора по единой метрике на уровне токенов и на одном корпусе в данной работе не проводилось: LLM-оценщик и классификатор запускались на разных корпусах (англоязычный RAGBench и русскоязычный RAGBench-RU соответственно) и в разных шкалах агрегации, поэтому сопоставление трактуется как сравнение с опорным ориентиром. Вместе с тем полученные результаты согласуются с ключевой идеей оригинальной статьи RAGBench о практической применимости компактного дообученного классификатора как существенно более дешёвой альтернативы LLM-оценщику.

6. Реализован прикладной сервис. Разработан Streamlit-сервис, поддерживающий одиночный и пакетный режимы применения обученных моделей, настройку

порогов и визуализацию токен-уровневой разметки. Сервис делает результаты работы доступными для прикладного использования, хотя не претендует на завершённую систему производственного развёртывания.

8.2 Границы применимости и дальнейшее развитие

Полученные результаты следует интерпретировать с учётом того, что RAGBench-RU построен машинным переводом исходного англоязычного корпуса, а перенесённая разметка не проходила полной независимой ручной верификации. Кроме того, сопоставление с LLM-оценщиком выполнялось как опорное: Qwen2.5-72B применялась к исходному англоязычному RAGBench, тогда как обученные классификаторы оценивались на RAGBench-RU.

Дальнейшее развитие работы может включать ручную проверку части русскоязычной разметки, запуск LLM-оценщика непосредственно на RAGBench-RU, расширение оценки на исходно русскоязычные корпуса и развитие сервиса для прикладной интеграции.

8.3 Артефакты работы

Открытые результаты работы включают:

- набор данных: <https://huggingface.co/datasets/CMCenjoyer/ragbench-ru>;
- опубликованные веса лучшей конфигурации оценщика (ModernBERT, длина 2048): <https://huggingface.co/CMCenjoyer/ru-trace-modernbert-2048>;
- исходный код обучающего конвейера и анализа: директория проекта;
- Streamlit-сервис: `new_streamlit.py`;
- таблицы и графики экспериментального анализа: `analysis_output/`;
- публикацию тезисов доклада по теме оценки качества RAG-систем в русскоязычном домене на конференции «Ломоносовские чтения - 2026» [9].

Список литературы

- [1] *Gao, Yunfan*. Retrieval-augmented generation for large language models: A survey / Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yixin Dai, Jiawei Sun, Haofen Wang, Haofen Wang // *arXiv preprint arXiv:2312.10997*. — 2023. — Vol. 2, no. 1.
- [2] *Es, Shahul*. Ragas: Automated evaluation of retrieval augmented generation / Shahul Es, Jithin James, Luis Espinosa Anke, Steven Schockaert // *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*. — 2024. — Pp. 150–158.
- [3] *Friel, Robert*. Ragbench: Explainable benchmark for retrieval-augmented generation systems / Robert Friel, Masha Belyi, Atindriyo Sanyal // *arXiv preprint arXiv:2407.11005*. — 2024.
- [4] *Belyi, Masha*. Luna: An evaluation foundation model to catch language model hallucinations with high accuracy and low cost / Masha Belyi, Robert Friel, Shuai Shao, Atindriyo Sanyal // *arXiv preprint arXiv:2406.00975*. — 2024.
- [5] *Saad-Falcon, Jon*. Ares: An automated evaluation framework for retrieval-augmented generation systems / Jon Saad-Falcon, Omar Khattab, Christopher Potts, Matei Zaharia // *arXiv preprint arXiv:2311.09476*. — 2023.
- [6] *Niu, Cheng*. Ragtruth: A hallucination corpus for developing trustworthy retrieval-augmented language models / Cheng Niu, Yuanhao Wu, Juno Zhu, Siliang Xu, Kashun Shum, Randy Zhong, Juntong Song, Tong Zhang // *arXiv preprint arXiv:2401.00396*. — 2023.
- [7] *Li, Yifei*. AttributionBench: How Hard is Automatic Attribution Evaluation? / Yifei Li, Xiang Yue, Zeyi Liao, Huan Sun // *arXiv preprint arXiv:2402.15089*. — 2024.
- [8] *Brown, Tom*. Language models are few-shot learners / Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell et al. // *Advances in neural information processing systems*. — 2020. — Vol. 33. — Pp. 1877–1901.

- [9] *Моисеев, Д. А.* Оценка качества RAG-систем в русскоязычном домене / Д. А. Моисеев, М. М. Тихомиров // Научная конференция «Ломоносовские чтения». Тезисы докладов, 23 марта — 5 апреля 2026 года. — Москва: МАКС Пресс, 2026. — Pp. 190–191.
- [10] *Kendall, Maurice G.* A new measure of rank correlation / Maurice G Kendall // *Biometrika*. — 1938. — Vol. 30, no. 1-2. — Pp. 81–93.
- [11] *He, Pengcheng.* Debertav3: Improving deberta using electra-style pre-training with gradient-disentangled embedding sharing / Pengcheng He, Jianfeng Gao, Weizhu Chen // *arXiv preprint arXiv:2111.09543*. — 2021.
- [12] *Hui, Binyuan.* Qwen2. 5-coder technical report / Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu et al. // *arXiv preprint arXiv:2409.12186*. — 2024.
- [13] *Qwen2 Technical Report* / An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, Guanting Dong, Haoran Wei, Huan Lin, Jialong Tang, Jialin Wang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Ma et al. // *arXiv preprint arXiv:2407.10671*. — 2024.
- [14] *Srivastava, Nitish.* Dropout: a simple way to prevent neural networks from overfitting / Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, Ruslan Salakhutdinov // *The journal of machine learning research*. — 2014. — Vol. 15, no. 1. — Pp. 1929–1958.
- [15] *Rumelhart, David E.* Learning representations by back-propagating errors / David E Rumelhart, Geoffrey E Hinton, Ronald J Williams // *nature*. — 1986. — Vol. 323, no. 6088. — Pp. 533–536.
- [16] *Hendrycks, Dan.* Gaussian error linear units (gelus) / Dan Hendrycks, Kevin Gimpel // *arXiv preprint arXiv:1606.08415*. — 2016.
- [17] *Chen, Jianlv.* BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation / Jianlv Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, Zheng Liu. — 2024.

- [18] *Spirin, Egor*. RuModernBERT: Modernized BERT for Russian / Egor Spirin, Boris Malashenko, Sokolov Andrey. — 2025.
<https://huggingface.co/deepvk/rumodernbert-base>.
- [19] *Su, Jianlin*. Roformer: Enhanced transformer with rotary position embedding / Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, Yunfeng Liu // *Neurocomputing*. — 2024. — Vol. 568. — P. 127063.

Приложение

A.1. Промпт для машинного перевода текстовых полей

Для перевода текстовых полей RAGBench-RU использовался единый минимальный промпт к модели Qwen2.5-72B-Instruct. Модель получала отдельный текстовый фрагмент и возвращала только его русский перевод без дополнительных рассуждений или комментариев.

SYSTEM:

Ты - профессиональный переводчик с английского на русский.

Переводи точно и понятно.

Рассуждения и дополнительные пояснения не нужны; верни только перевод.

USER:

{text}

Сохранение структуры примеров не задавалось отдельным JSON-проммптом, а обеспечивалось на уровне конвейера: текстовые поля переводились отдельно, ключи предложений сохранялись из исходного корпуса, после чего выполнялись структурные проверки числа предложений и наличия ключей разметки.

A.2. Промпт LLM-оценщика без дообучения (Qwen2.5-72B)

Промпт, использованный в сравнительном эксперименте оценки без дообучения на базе Qwen2.5-72B (см. раздел 6.7), соответствует англоязычной схеме промпта из оригинальной работы RAGBench. Оценщик не используется для построения разметки корпуса RAGBench-RU.

Модели передавались документы с ключами предложений, вопрос и ответ. На выходе запрашивался JSON-объект с полями `relevance_explanation`, `all_relevant_sentence_keys`, `overall_supported_explanation`, `overall_supported_sentence_support_information` и `all_utilized_sentence_keys`.

Поле `all_relevant_sentence_keys` содержит ключи предложений документов, релевантных вопросу. Поле `all_utilized_sentence_keys` содержит ключи

предложений документов, использованных при построении ответа. Поле `overall_supported` отражает, поддерживается ли ответ в целом документами, а `sentence_support_information` содержит информацию о поддержке отдельных предложений ответа.

В последующем количественном анализе использовались только поля `all_relevant_sentence_keys` и `all_utilized_sentence_keys`; поле `overall_supported` и подробная информация `sentence_support_information` не входили в метрики таблицы 9. Полный текст промпта приведён в ноутбуке `test_llm_razm.ipynb`.

A.3. Статистика ошибок LLM-оценщика по поддатасетам

Таблица 9 - агрегаты ошибок разметки Qwen2.5-72B (совпадение, пропуски, лишние ключи) на тестовых поддатасетах исходного англоязычного RAGBench. *overlap* - доля ключей, совпавших с эталоном; *miss* - среднее число пропущенных ключей на пример; *extra* - среднее число лишних. Метрики получены при запуске LLM-оценщика на англоязычных примерах с оригинальным англоязычным промптом из статьи RAGBench [3]; русскоязычные данные RAGBench-RU на вход большой языковой модели не подавались. Соответственно, эти значения служат опорным ориентиром при сравнении с обученным классификатором, оценённым на русскоязычном RAGBench-RU, и не являются результатом контролируемого эксперимента на одном и том же корпусе.

Таблица 9: Поддатасетные метрики LLM-оценщика Qwen2.5-72B (исходный англоязычный RAGBench)

поддатасет	rel. overlap	rel. miss	rel. extra	util. overlap	util. miss	util. extra
hotpotqa	0.74	0.87	1.12	0.85	0.41	0.68
hagrid	0.68	1.25	1.34	0.81	0.55	0.73
expertqa	0.65	1.43	1.51	0.79	0.60	0.81
covidqa	0.71	0.94	1.08	0.84	0.44	0.67
delucionqa	0.76	0.71	0.95	0.87	0.38	0.61
cuad	0.62	1.68	1.82	0.76	0.72	0.94
finqa	0.72	0.99	1.15	0.85	0.43	0.69
среднее	0.70	1.12	1.28	0.83	0.50	0.73

Наибольшая доля избыточной генерации наблюдается в **cuad** (юридический домен) и **expertqa** (научный домен), где LLM склонна добавлять «похоже релевантные» предложения в разметку.

А.4. Полная сетка конфигураций

Полная таблица всех ~ 80 обученных конфигураций (базовая модель, длина, веса, язык, голова, F1 на уровне токенов по таргетам) доступна в файле `analysis_output/tables/full_grid.csv`. В настоящем тексте приведены только основные агрегаты; CSV распространяется вместе с кодом работы.

А.5. Пример конфига эксперимента

```
# model_utils/config.py (fragment)
@dataclass
class DataConfig:
    question_key: str = "question_ru"
    documents_sentences_key: str = "documents_sentences_ru"
    response_key: str = "response_ru"
    relevant_keys_field: str = "all_relevant_sentence_keys"
    utilized_keys_field: str = "all_utilized_sentence_keys"
    adherence_field: str = "adherence_score"
```

```

        max_length:                int = 2048

@dataclass
class TrainingConfig:
    num_epochs:                    int    = 4
    train_batch_size:              int    = 64
    eval_batch_size:               int    = 32
    backbone_lr:                   float  = 2e-5
    head_lr:                       float  = 3e-4
    weight_decay:                  float  = 1e-3
    grad_clip:                     float  = 1.0
    warmup_ratio:                  float  = 0.1
    loss_weights:                  Tuple[float,float,float] = (3.0, 3.0, 1.0)
    amp:                           bool   = True

```